

DOCUMENT DE CONCEPTION

SchoolManage App

*Architecture Micro-services • Specifications Fonctionnelles • Cas d'Utilisation •
Diagrammes • Reference Technique*

Version du document : 1.0

Date : Juillet 2026

Type de document : Dossier de Conception Generale et Detaillee (DCG/DCD)

Stack technique : Java Spring Boot, Python (FastAPI/Django), Node.js, React Native, React.js,
PostgreSQL, Oracle, Redis, Kafka/RabbitMQ

Cadre : Projet academique

Table des Matieres

1. Introduction et Contexte du Projet

1.1 Presentation de la Plateforme SchoolManage App

1.2 Objectifs

1.3 Perimetre Fonctionnel

1.4 Stack Technologique Globale

2. Identification des Acteurs

2.1 Cas d'Utilisation par Acteur - Synthese

2.2 Diagramme de Cas d'Utilisation

3. Description Textuelle des Cas d'Utilisation

3.1 Gestion des Comptes | 3.2 Inscription Scolaire | 3.3 Paiement

3.4 Presence | 3.5 Pedagogie & Bulletins | 3.6 Communication | 3.7 Supervision

4. Tableau de Regroupement Fonctionnel

5. Description Detaillee des Micro-services

5.1 Micro-services non fonctionnels (S1-S3)

5.2 Micro-services metiers (S4-S15)

6. Architecture Logicielle

6.1 Vue Globale | 6.2 Communications Inter-services | 6.3 Strategies Techniques

7. Diagrammes de Conception

7.1 Diagrammes de Classes | 7.2 Diagrammes de Sequence | 7.3 Diagrammes d'Activite

7.4 Diagrammes d'Etat-Transition | 7.5 Diagrammes de Composants

8. Reference Technique des Services

8.1 Conventions de nommage | 8.2 Ports | 8.3 Bases de donnees

8.4 Redis | 8.5 Kafka/RabbitMQ | 8.6 Environnements

9. Deploiement, Conteneurisation et Monitoring

10. Backlog Initial des User Stories

11. Exigences Non Fonctionnelles

12. Glossaire

13. Conclusion

1. Introduction et Contexte du Projet

1.1 Presentation de la Plateforme SchoolManage App

SchoolManage App est une plateforme numerique de gestion scolaire au Cameroun. Elle vise a moderniser et digitaliser l'ensemble de la chaine de valeur de la gestion d'un etablissement scolaire, depuis l'inscription des eleves et le paiement des frais jusqu'au suivi pedagogique et a la communication avec les familles.

La plateforme adopte une architecture micro-services permettant une evolutivite independante de chaque composant fonctionnel, une haute disponibilite et une maintenabilite optimale du systeme.

1.2 Objectifs

- Permettre aux parents de payer les frais de scolarite en ligne (Mobile Money) et de suivre la scolarite de leurs enfants
- Offrir aux etablissements un outil complet de gestion operationnelle (inscriptions, personnel, classes)
- Automatiser le suivi de presence des eleves et du personnel (check-in/out par QR code)
- Digitaliser la production des bulletins et des cartes d'identite scolaires
- Fluidifier la communication ecole-famille via l'integration WhatsApp et les notifications
- Fournir aux directeurs des tableaux de bord et rapports de performance
- Assurer une supervision globale multi-etablissement par l'administrateur systeme

1.3 Perimetre Fonctionnel

La plateforme SchoolManage App couvre 12 domaines fonctionnels metier, chacun implemente sous forme d'un micro-service independant, auxquels s'ajoutent 3 micro-services non fonctionnels d'infrastructure.

#	Micro-service	Domaine fonctionnel
S4	auth-service	Authentification & Autorisations (RBAC / JWT)
S5	registration-service	Inscription des eleves et du personnel
S6	payment-service	Paieement des frais / salaires (Mobile Money)
S7	presence-service	Presence & assidue (check-in QR)
S8	notification-service	Notifications Email / SMS / Push
S9	reportcard-service	Notes et bulletins scolaires
S10	userstatus-service	Statuts utilisateurs en temps reel
S11	whatsapp-service	Communaute WhatsApp (parents-ecole)
S12	pedagogic-service	Ressources et formation pedagogique
S13	schoolid-service	Cartes d'identite scolaires (QR code)
S14	analytics-service	Tableaux de bord et indicateurs (KPI)
S15	admin-service	Configuration multi-etablissement

1.4 Stack Technologique Globale

Couche	Technologie	Usage
Client mobile	React Native	Application iOS / Android (parents, enseignants)
Client web	React.js	Tableau de bord web (admin, directeur)
Backend (Java)	Spring Boot	auth-service, registration-service, payment-service, admin-service
Backend (Python)	FastAPI / Django	reportcard-service, pedagogic-service, schoolid-service, analytics-service
Backend (Node.js)	Express.js	presence-service, notification-service, userstatus-service, whatsapp-service
Base de donnees	PostgreSQL	Donnees relationnelles transactionnelles

		(pattern Database per Service)
Reporting / Archives	Oracle	Rapports d'entreprise, archives financieres pluriannuelles
Cache / Verrous	Redis	Sessions JWT, cache de lecture, verrous anti-doublon
Messagerie async	Kafka / RabbitMQ	Communication inter-services par evenements
Stockage objets	MinIO / AWS S3	Documents, photos, PDF (recus, bulletins, cartes ID)
API Gateway	Spring Cloud Gateway / Kong	Routage et validation JWT centralisee
Discovery / Config	Eureka / Spring Cloud Config	Decouverte de services et configuration centralisee

2. Identification des Acteurs

Le tableau ci-dessous recense l'ensemble des acteurs - humains et systemes externes - interagissant avec la plateforme SchoolManage App.

Acteur	Type	Description
Parent	Humain	Consulte la scolarite de ses enfants, paie les frais, recoit les notifications et communique avec l'ecole.
Eleve	Humain	Effectue son check-in de presence, consulte ses notes et son bulletin.
Enseignant	Humain	Saisit les notes, marque la presence, publie des annonces et gere les ressources pedagogiques.
Administrateur / Directeur	Humain	Gere l'etablissement : inscriptions, personnel, tarification, tableaux de bord.
Administrateur Systeme	Humain	Supervise la plateforme globale : etablissements, comptes, configuration technique.
Systeme de paiement	Systeme externe	Service Mobile Money (MTN MoMo / Orange Money) traitant les transactions financieres.
Service Email / SMS / Push	Systeme externe	Gere l'envoi automatique des notifications (SendGrid, Twilio, FCM).
WhatsApp Cloud API	Systeme externe	Gere la messagerie de groupe et les annonces vers les parents.

2.1 Cas d'Utilisation par Acteur - Synthese

Parent

- S'inscrire, se connecter, se deconnecter, gerer son profil
- Payer les frais de scolarite (Mobile Money) et consulter les recus
- Consulter la presence, les notes et le bulletin de ses enfants
- Recevoir des notifications (annonces, retards, paiements)
- Communiquer avec l'ecole via WhatsApp

Enseignant

- Se connecter, gerer son profil
- Marquer la presence, saisir les notes
- Publier une annonce, uploader une ressource pedagogique
- Publier / consulter un statut (absence, deplacement)

Administrateur / Directeur

- Inscrire un eleve ou un membre du personnel
- Generer les cartes d'identite et les bulletins
- Consulter le tableau de bord et les statistiques de l'etablissement
- Gerer les paiements de salaires et les rapports financiers
- Diffuser des annonces globales

Administrateur Systeme

- Ajouter / configurer un etablissement

- Gerer les comptes utilisateurs (activation, suspension)
- Visualiser l'activite globale et les statistiques multi-etablissement

2.2 Diagramme de Cas d'Utilisation

Diagramme de Cas d'Utilisation - SchoolManage App

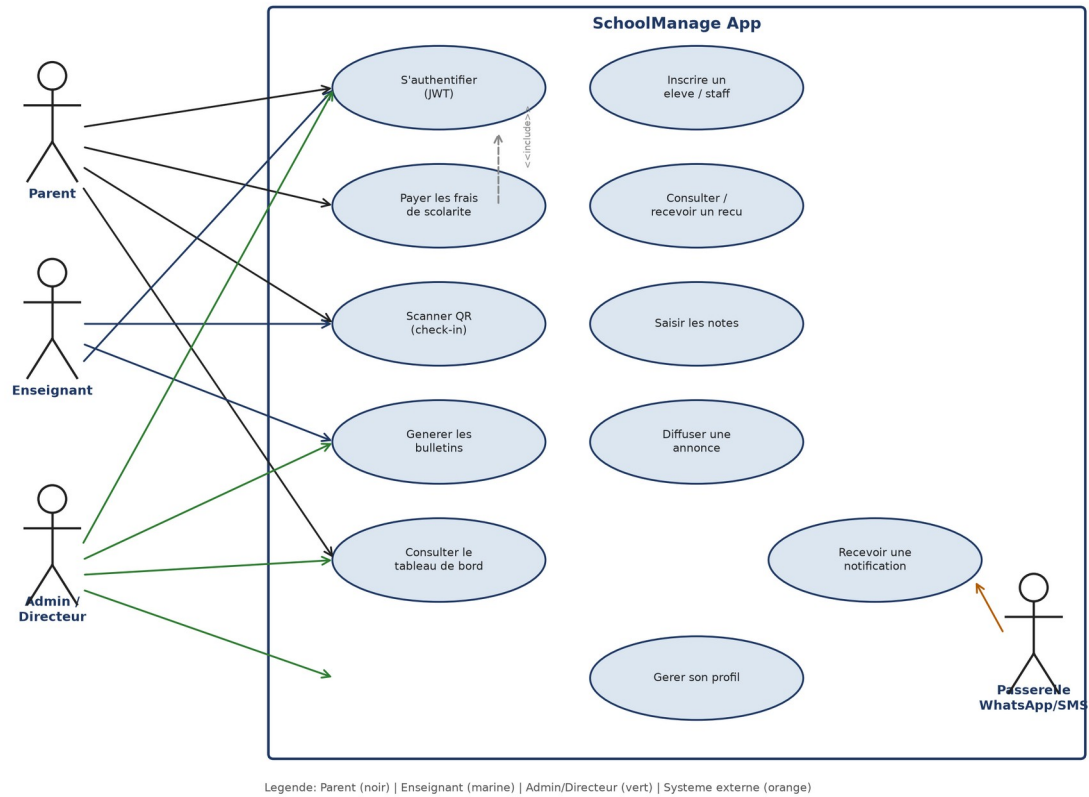


Figure 1 - Diagramme de cas d'utilisation

3. Description Textuelle des Cas d'Utilisation

Chaque fiche ci-dessous detaille un cas d'utilisation cle avec ses acteurs, preconditions et postconditions.

3.1 Gestion des Comptes

Identifiant	UC1
Titre	S'inscrire
Acteur principal	Parent / Enseignant
Acteurs secondaires	Service email / notifications
Description	Permet a un utilisateur de creer un compte en fournissant ses informations personnelles (nom, prenom, email, telephone, mot de passe). Le systeme verifie l'unicite de l'email.
Precondition	Acceder a l'application SchoolManage.
Postcondition	Compte cree et actif.

Identifiant	UC2
Titre	Se connecter
Acteur principal	Tous acteurs
Description	L'utilisateur s'authentifie via email/mot de passe. Un token JWT est genere (access token 15 min + refresh token 7 jours) encode avec le role et les droits.
Precondition	Avoir un compte actif.
Postcondition	JWT genere, session active.

Identifiant	UC3
Titre	Reinitialiser le mot de passe
Acteur principal	Tous acteurs
Acteurs secondaires	Service email / notifications
Description	L'utilisateur demande un lien de reinitialisation par email. Apres validation du lien tokenise (TTL 1h), il saisit un nouveau mot de passe. Les anciens tokens sont revoques.
Precondition	Avoir un compte actif.
Postcondition	Mot de passe modifie, anciens tokens invalides.

Identifiant	UC4
Titre	Se deconnecter
Acteur principal	Tous acteurs
Description	L'utilisateur se deconnecte. Le token JWT actuel est ajoute a la blacklist Redis pour empecher toute reutilisation avant expiration.
Precondition	Etre connecte.
Postcondition	Token invalide dans Redis.

Identifiant	UC5
Titre	Gerer son profil
Acteur principal	Tous acteurs
Description	L'utilisateur consulte et met a jour ses informations personnelles (photo, telephone, adresse).
Precondition	Etre connecte.
Postcondition	Profil mis a jour.

3.2 Inscription Scolaire

Identifiant	UC6
-------------	-----

Titre	Inscrire un eleve / staff
Acteur principal	Administrateur
Acteurs secondaires	Auth Service, Notification Service
Description	Permet a l'administrateur d'enregistrer un nouvel eleve ou membre du personnel avec documents (photo, acte de naissance) et affectation de classe/departement.
Precondition	Etre connecte avec les droits appropries.
Postcondition	Compte cree, dossier enregistre, notification de bienvenue envoyee.

Identifiant	UC7
Titre	Generer la carte d'identite scolaire
Acteur principal	Systeme (declenche par UC6)
Description	A l'issue de l'inscription, le systeme genere automatiquement une carte d'identite avec QR code integre et photo de l'eleve/staff.
Precondition	Inscription validee.
Postcondition	Carte ID generee et disponible en telechargement (PDF).

Identifiant	UC8
Titre	Affecter une classe
Acteur principal	Administrateur
Description	Permet d'affecter ou de modifier la classe d'un eleve inscrit.
Precondition	Eleve inscrit, classe existante.
Postcondition	Affectation mise a jour.

3.3 Paiement

Identifiant	UC9
Titre	Payer les frais de scolarite
Acteur principal	Parent
Acteurs secondaires	Systeme de paiement, Notification Service
Description	Le parent choisit les frais a regler et la methode de paiement (MTN MoMo / Orange Money). Le service initialise la transaction avec une cle d'idempotence, attend la confirmation du webhook operateur et met a jour le statut.
Precondition	Etre connecte. Solde Mobile Money suffisant.
Postcondition	Transaction validee ou refusee. Recu et notification envoyes.

Identifiant	UC10
Titre	Consulter un recu de paiement
Acteur principal	Parent
Description	Permet au parent de telecharger le recu PDF d'un paiement effectue.
Precondition	Paieement confirme.
Postcondition	Recu PDF telecharge.

Identifiant	UC11
Titre	Effectuer un paiement de salaire
Acteur principal	Administrateur
Description	L'administrateur declenche le versement du salaire d'un membre du personnel pour une periode donnee.
Precondition	Etre connecte avec les droits appropries.
Postcondition	Paieement enregistre (status=COMPLETED), notification envoyee.

Identifiant	UC12
Titre	Consulter l'historique de paiement
Acteur principal	Parent / Administrateur
Description	Affiche l'historique complet des paiements effectues pour un eleve ou un etablissement.
Precondition	Etre connecte.
Postcondition	Liste des paiements affichee.

3.4 Presence

Identifiant	UC13
Titre	Scanner QR (check-in / check-out)
Acteur principal	Eleve / Enseignant
Description	L'utilisateur scanne le QR code de sa carte pour enregistrer son entree ou sa sortie de l'etablissement.
Precondition	Etre inscrit, carte ID valide.
Postcondition	Presence enregistree (check_in / check_out), statut mis a jour dans Redis.

Identifiant	UC14
Titre	Consulter le tableau de presence
Acteur principal	Enseignant / Administrateur
Description	Affiche le tableau d'assiduite quotidien/hebdomadaire d'une classe ou d'un etablissement.
Precondition	Etre connecte.
Postcondition	Tableau de presence affiche.

Identifiant	UC15
Titre	Recevoir une alerte d'absence
Acteur principal	Parent
Acteurs secondaires	Notification Service
Description	Le systeme detecte automatiquement les absences non justifiees et notifie le parent par SMS/push.
Precondition	Aucun check-in enregistre a la cloture de la journee.
Postcondition	Notification d'absence envoyee au parent.

3.5 Pedagogie & Bulletins

Identifiant	UC16
Titre	Saisir les notes
Acteur principal	Enseignant
Description	Permet a l'enseignant de saisir les notes d'un eleve par matiere et par periode.
Precondition	Etre connecte, eleve inscrit dans la classe.
Postcondition	Note enregistree.

Identifiant	UC17
Titre	Generer les bulletins
Acteur principal	Administrateur
Acteurs secondaires	Notification Service
Description	Declenche le calcul des moyennes et des rangs, puis la generation du PDF du bulletin pour une classe et une periode donnees.
Precondition	Toutes les notes de la periode saisies.
Postcondition	Bulletins generes et stocks, notification de disponibilite

	envoyee.
--	----------

Identifiant	UC18
Titre	Consulter le bulletin
Acteur principal	Parent / Eleve
Description	Permet de consulter ou telecharger le bulletin scolaire au format PDF.
Precondition	Bulletin publie.
Postcondition	Bulletin PDF telecharge.

3.6 Communication

Identifiant	UC19
Titre	Diffuser une annonce
Acteur principal	Enseignant / Administrateur
Acteurs secondaires	WhatsApp Cloud API, Notification Service
Description	Permet de diffuser un message (retard, evenement, information) a un groupe WhatsApp de classe ou via notification push/SMS.
Precondition	Etre connecte.
Postcondition	Message diffuse aux destinataires concernes.

Identifiant	UC20
Titre	Publier / consulter un statut utilisateur
Acteur principal	Tous acteurs
Description	Un utilisateur publie un statut (malade, en deplacement, distanciel) visible en temps reel par l'administration.
Precondition	Etre connecte.
Postcondition	Statut mis a jour et diffuse (WebSocket).

Identifiant	UC21
Titre	Recevoir une notification
Acteur principal	Tous acteurs
Description	L'utilisateur recoit une notification (email, SMS ou push) suite a un evenement de la plateforme (paiement, bulletin, annonce, absence).
Precondition	Evenement declencheur emis.
Postcondition	Notification livree et journalisee.

3.7 Supervision

Identifiant	UC22
Titre	Consulter le tableau de bord analytique
Acteur principal	Directeur / Administrateur
Description	Affiche les indicateurs cles (effectifs, taux de presence, recettes, performance academique) sous forme de graphiques.
Precondition	Etre connecte.
Postcondition	Tableau de bord affiche (donnees en cache 1h).

Identifiant	UC23
Titre	Gerer la configuration multi-etablissement
Acteur principal	Administrateur Systeme
Description	Permet d'ajouter un etablissement, de configurer ses parametres et de gerer les comptes utilisateurs a l'echelle de la

	plateforme.
Precondition	Etre connecte en tant qu'administrateur systeme.
Postcondition	Etablissement / configuration cree ou mis a jour.

4. Tableau de Regroupement Fonctionnel

Les cas d'utilisation sont regroupés par domaine fonctionnel pour faciliter la planification du développement.

Domaine fonctionnel	Cas d'utilisation
Gestion des comptes	S'inscrire - Se connecter / Se déconnecter - Reinitialiser mot de passe - Gérer son profil
Inscription scolaire	Inscrire un élève/staff - Générer la carte ID - Affecter une classe
Paie	Payer les frais (Mobile Money) - Consulter un reçu - Payer un salaire - Historique des paiements
Présence	Scanner QR (check-in/out) - Consulter le tableau de présence - Recevoir une alerte d'absence
Pédagogie & Bulletins	Saisir les notes - Générer les bulletins - Consulter le bulletin
Communication	Diffuser une annonce - Publier/consulter un statut - Recevoir une notification
Supervision	Consulter le tableau de bord - Gérer la configuration multi-établissement

5. Description Detaillee des Micro-services

Chaque micro-service est decrit avec ses responsabilites, ses endpoints REST, ses communications avec les autres services et ses cas d'utilisation associes.

5.1 Micro-services Non Fonctionnels

Les micro-services non fonctionnels constituent l'infrastructure technique de la plateforme. Ils ne traitent pas directement de donnees metier, mais garantissent la decouverte des services, la centralisation de la configuration et le routage intelligent des requetes. Aucun service metier ne peut demarrer correctement sans que ces trois services soient operationnels.

Ordre de demarrage obligatoire : sm-config-service (8080) -> sm-registry-service (8761) -> sm-gateway-service (8888) -> services metier en parallele

S1 - sm-config-service

Stack technique : Spring Cloud Config Server — Port 8080 — GitHub (schoolmanage-cloud-config)

Le sm-config-service est le referentiel central de configuration de toute la plateforme SchoolManage. Il implemente le patron Externalized Configuration : au lieu d'embarquer les parametres de connexion (base de donnees, Redis, Kafka, ports) dans chaque service, tous ces parametres sont stockes dans un depot Git dedie et distribues dynamiquement a chaque service au demarrage. En cas d'indisponibilite du config-service, chaque service bascule sur ses valeurs locales par default (degradation gracieuse).

Responsabilites

- Distribuer la configuration centralisee (ports, bases, Redis, Kafka) a tous les services
- Lire les fichiers de configuration depuis le depot Git dedie
- Supporter plusieurs profils d'environnement (dev, test, prod)
- Permettre la mise a jour dynamique de configuration sans redemarrage
- Fournir un endpoint de sante /actuator/health

S2 - sm-registry-service

Stack technique : Spring Cloud Netflix Eureka — Port 8761

Le sm-registry-service est le registre de decouverte de services (Service Registry) de la plateforme. Chaque service metier s'enregistre aupres de ce registre au demarrage. Le Gateway interroge le registry pour connaitre les instances disponibles d'un service cible, puis route la requete vers l'une d'elles (load balancing cote client). En cas de panne d'une instance, le registry l'expulse automatiquement apres expiration de son bail (90 secondes par default).

Responsabilites

- Recevoir et stocker les enregistrements de tous les services (heartbeat 30s)
- Maintenir la liste des instances actives par service
- Expulser automatiquement les instances inactives (bail expire)
- Exposer la liste des instances au gateway pour le load balancing
- Fournir un tableau de bord visuel de supervision en temps reel

S3 - sm-gateway-service

Stack technique : Spring Cloud Gateway / Kong — Port 8888 — JWT / RBAC

Le sm-gateway-service est la porte d'entree unique de toute la plateforme. Il implemente le patron API Gateway : aucun client (mobile, web) ne communique directement avec les services metier — toutes les requetes transitent obligatoirement par ce service. Il assure trois responsabilites fondamentales : (1) le routage des requetes vers le bon service metier, (2) la validation du token JWT et le controle d'accès RBAC, et (3) le load balancing entre les instances d'un meme service.

Responsabilites

- Routage de toutes les requetes entrantes vers les services metier appropries
- Validation du token JWT et verification des droits RBAC sur chaque requete
- Isolation automatique des instances defaillantes (circuit breaking)
- Propagation des headers d'authentification en aval (X-User-Id, X-User-Role)
- Limitation de debit (rate limiting) et protection contre les attaques

5.2 Micro-services Metier

S4 - auth-service : Authentification et Autorisation

Stack technique : Java Spring Boot + Keycloak (OAuth2/OIDC) — Port 8081 — PostgreSQL, Redis

Description

Le auth-service est le gardien central de la plateforme. Il gere l'ensemble du cycle de vie des identites : inscription, connexion, generation et validation des tokens JWT, revocation et controle d'accès basé sur les rôles (RBAC). Le service supporte les rôles Parent, Enseignant, Administrateur et Directeur, chacun avec des permissions spécifiques encodées dans le payload JWT.

Responsabilités

- Inscription et vérification de l'unicité de l'email
- Authentification multi-rôles avec génération de JWT (access 15min + refresh 7j)
- Blacklist des tokens invalides dans Redis
- Contrôle d'accès RBAC sur toutes les routes protégées
- Reinitialisation sécurisée de mot de passe (lien tokenisé, TTL 1h)
- Désactivation / suspension de comptes par l'administrateur

Endpoints REST

Methode	Route	Rôles autorisés	Description
POST	/api/auth/register	Public	Créer un nouveau compte
POST	/api/auth/login	Public	Authentifier et retourner les tokens JWT
POST	/api/auth/logout	Authentifié	Invalidiser le token (blacklist Redis)
POST	/api/auth/refresh	Authentifié	Renouveler l'accès token
GET	/api/auth/me	Authentifié	Retourner le profil de l'utilisateur connecté
POST	/api/auth/validate-token	Services internes	Valider un JWT (usage interne)
PATCH	/api/auth/users/:id/status	Admin	Activer, désactiver ou suspendre un compte

Communications inter-services

Service cible	Protocole / Evenement	Payload	Declencheur
notification-service	Kafka / user.registered	{email, nom, token}	Inscription -> email de bienvenue
notification-service	Kafka / password.reset	{email, reset_link}	Demande de réinitialisation
registration-service	Kafka / user.created	{userId, email, role}	Création de profil initial
Gateway	REST / HTTP	{Authorization: Bearer}	Validation de chaque requête entrante

Cas d'utilisation associés

ID	Cas d'utilisation
UC1	S'inscrire
UC2	Se connecter
UC3	Reinitialiser le mot de passe
UC4	Se déconnecter

S5 - registration-service : Inscription des Elèves et du Personnel

Stack technique : Java Spring Boot — Port 8082 — PostgreSQL, MinIO

Description

Le registration-service gere l'enregistrement des eleves et du personnel : formulaire d'inscription, upload de documents (photo, acte de naissance), affectation de classe et historique d'inscription. Il declenche la creation du compte via auth-service et la generation de la carte d'identite via schoolid-service.

Responsabilites

- Enregistrement des eleves et onboarding du personnel
- Upload et stockage des documents (MinIO)
- Logique d'affectation automatique de classe
- Suivi de l'historique des inscriptions
- Declenchement de la generation de carte ID

Endpoints REST

Methode	Route	Roles autorises	Description
POST	/api/v1/registrations/student	Admin	Inscrire un nouvel eleve
POST	/api/v1/registrations/staff	Admin	Inscrire un membre du personnel
GET	/api/v1/registrations/{id}	Admin	Consulter un dossier d'inscription
PATCH	/api/v1/registrations/{id}/class	Admin	Affecter / modifier la classe

Communications inter-services

Service cible	Protocole / Evenement	Payload	Declencheur
auth-service	REST / HTTP	{email, role}	Creation du compte utilisateur
schoolid-service	Kafka / student.enrolled	{studentId, photo}	Inscription validee -> generation ID
notification-service	Kafka / student.enrolled	{studentId, parentEmail}	Envoi notification de bienvenue
whatsapp-service	Kafka / student.enrolled	{classId, studentId}	Ajout au groupe WhatsApp de classe

Cas d'utilisation associes

ID	Cas d'utilisation
UC6	Inscrire un eleve / staff
UC8	Affecter une classe

S6 - payment-service : Service de Paiement

Stack technique : Java Spring Boot — Port 8083 — PostgreSQL + Oracle, Kafka (idempotency keys)

Description

Le payment-service orchestre toutes les transactions financieres de la plateforme : frais de scolarite et salaires. Il integre les API Mobile Money de MTN (MoMo) et Orange Money. Chaque transaction est securisee par un mecanisme d'idempotence (cle unique Redis) qui evite les doubles debits en cas de retry reseau. Le service expose des webhooks pour recevoir les confirmations des operateurs.

Responsabilites

- Initiation des paiements Mobile Money (MTN MoMo / Orange Money)
- Reception et traitement des webhooks de confirmation / echec
- Mecanisme d'idempotence pour eviter les doubles debits
- Gestion des remboursements en cas d'annulation eligible
- Generation automatique des recus PDF
- Calcul des recapitulatifs financiers par etablissement

Endpoints REST

Methode	Route	Roles autorises	Description
POST	/api/v1/payments/fees	Parent	Initier un paiement de frais de scolarite
POST	/api/v1/payments/salary	Admin	Initier un paiement de salaire
GET	/api/v1/payments/{id}/status	Parent / Admin	Consulter le statut d'une transaction
GET	/api/v1/payments/{id}/receipt	Parent	Telecharger le reçu PDF
POST	/api/v1/payments/webhook/mtn	Systeme (MTN)	Notification de paiement MTN
POST	/api/v1/payments/webhook/orange	Systeme (Orange)	Notification de paiement Orange
GET	/api/v1/payments/student/{id}	Parent / Admin	Historique des paiements d'un eleve
GET	/api/v1/payments/reports/revenue	Admin / Directeur	Rapport de recettes

Communications inter-services

Service cible	Protocole / Evenement	Payload	Declencheur
notification-service	Kafka / payment.completed	{userId, montant, recu_url}	Paiement reussi -> recu au parent
notification-service	Kafka / payment.failed	{userId, motif}	Paiement echoue -> notification
analytics-service	Kafka / payment.completed	{montant, etablisementId}	Mise a jour des indicateurs financiers

Cas d'utilisation associes

ID	Cas d'utilisation
UC9	Payer les frais de scolarite
UC10	Consulter un recu de paiement
UC11	Effectuer un paiement de salaire
UC12	Consulter l'historique de paiement

S7 - presence-service : Presence et Assidue

Stack technique : Node.js / Express — Port 8084 — PostgreSQL, Redis

Description

Le presence-service gere le pointage des eleves et du personnel par QR code ou biometrie. Les disponibilites et statuts du jour sont maintenus dans Redis pour un affichage temps reel. Un job planifie detecte quotidiennement les absences non justifiees et declenche une alerte.

Responsabilites

- Enregistrement des check-in / check-out par QR code
- Tableau de bord de presence en temps reel
- Detection automatique des absences (job planifie)
- Suivi des retards et generation de rapports journaliers/hebdomadaires

Endpoints REST

Methode	Route	Roles autorises	Description
POST	/api/v1/presence/check-in	Eleve / Staff	Enregistrer une entree
POST	/api/v1/presence/check-out	Eleve / Staff	Enregistrer une sortie
GET	/api/v1/presence/class/{id}	Enseignant / Admin	Tableau de presence d'une classe
GET	/api/v1/presence/student/{id}	Parent / Admin	Historique de presence d'un eleve

Communications inter-services

Service cible	Protocole / Evenement	Payload	Declencheur
notification-service	Kafka /	{studentId, date}	Absence detectee -> alerte parent

	presence.absence.detected		
analytics-service	Kafka / presence.recorded	{status, classId}	Mise a jour des indicateurs d'assiduite

Cas d'utilisation associes

ID	Cas d'utilisation
UC13	Scanner QR (check-in/out)
UC14	Consulter le tableau de presence
UC15	Recevoir une alerte d'absence

S8 - notification-service : Service de Notifications

Stack technique : Node.js / Express — Port 8085 — Kafka consumer, Redis Pub/Sub

Description

Le notification-service est le service de communication sortante de la plateforme. Il s'abonne aux evenements publies par les autres services via Kafka et declenche l'envoi de communications par email (SendGrid), SMS (Twilio) ou push (FCM). Chaque notification est journalisee avec son statut (envoyee, echec, retry).

Responsabilites

- Envoi d'emails transactionnels (confirmation, reçu, bulletin)
- Notifications push et SMS (alertes, retards, annonces)
- Journalisation de chaque notification envoyee
- Retry automatique en cas d'echec d'envoi (backoff exponentiel)

Endpoints REST

Methode	Route	Roles autorises	Description
POST	/api/v1/notifications/send	Services internes / Admin	Envoyer une notification manuelle
GET	/api/v1/notifications/user/{id}	Utilisateur	Historique des notifications recues
GET	/api/v1/notifications/logs	Admin	Consulter les logs d'envoi (statuts, erreurs)
PATCH	/api/v1/notifications/{id}/retry	Admin	Forcer le renvoi d'une notification en echec

Communications inter-services

Service cible	Protocole / Evenement	Payload	Declencheur
-	Kafka (consumer multi-topics)	evenements de tous les services	Reaction a tout evenement metier

Cas d'utilisation associes

ID	Cas d'utilisation
UC19	Diffuser une annonce
UC21	Recevoir une notification

S9 - reportcard-service : Notes et Bulletins

Stack technique : Python FastAPI — Port 8086 — PostgreSQL, WeasyPrint/ReportLab

Description

Le reportcard-service gere la saisie des notes par matiere et par periode, le calcul automatique des moyennes et des rangs, ainsi que la generation des bulletins au format PDF. Les bulletins generes sont stockes sur MinIO et accessibles en lecture par les parents.

Responsabilités

- Saisie des notes par matiere et par periode
- Calcul automatique des moyennes ponderees et des rangs
- Generation PDF des bulletins (WeasyPrint/ReportLab)
- Archives historiques des notes par eleve

Endpoints REST

Methode	Route	Roles autorises	Description
POST	/api/v1/reports/grades	Enseignant	Saisir une note
POST	/api/v1/reports/generate	Admin	Generer les bulletins d'une classe/periode
GET	/api/v1/reports/student/{id}	Parent / Eleve	Consulter / telecharger le bulletin
GET	/api/v1/reports/class/{id}/summary	Enseignant / Admin	Statistiques de classe

Communications inter-services

Service cible	Protocole / Evenement	Payload	Declencheur
notification-service	Kafka / reportcard.available	{studentId, bulletin_url}	Bulletin genere -> notification
analytics-service	Kafka / reportcard.generated	{classId, moyennes}	Mise a jour indicateurs academiques

Cas d'utilisation associes

ID	Cas d'utilisation
UC16	Saisir les notes
UC17	Generer les bulletins
UC18	Consulter le bulletin

S10 - userstatus-service : Statuts Utilisateurs

Stack technique : Node.js / WebSocket — Port 8087 — Redis

Description

Le userstatus-service permet a un eleve ou un membre du personnel de publier son statut courant (malade, en deplacement, distanciel). Les changements de statut sont diffuses en temps reel via WebSocket vers les tableaux de bord d'administration.

Responsabilites

- Publication et mise a jour du statut utilisateur
- Diffusion temps reel des changements (WebSocket)
- Historique des statuts par utilisateur

Endpoints REST

Methode	Route	Roles autorises	Description
POST	/api/v1/status	Tous acteurs	Publier un statut
GET	/api/v1/status/history/{id}	Admin	Consulter l'historique de statut
GET	/api/v1/status/live	Admin	Flux temps reel des statuts (WebSocket)

Communications inter-services

Service cible	Protocole / Evenement	Payload	Declencheur
notification-service	Kafka / status.changed	{userId, statusType}	Changement de statut -> notification

Cas d'utilisation associes

ID	Cas d'utilisation
UC20	Publier / consulter un statut utilisateur

S11 - whatsapp-service : Communauté WhatsApp

Stack technique : Node.js — Port 8088 — WhatsApp Cloud API, MongoDB

Description

Le whatsapp-service integre l'API WhatsApp Business (Cloud API) pour la creation automatique de groupes de classe lors des inscriptions, la diffusion d'annonces et la messagerie parent-enseignant. Les messages sont archives dans MongoDB pour moderation.

Responsabilites

- Creation automatique d'un groupe WhatsApp par classe
- Diffusion d'annonces vers les groupes de classe
- Messagerie directe parent-enseignant
- Archivage et moderation des messages

Endpoints REST

Methode	Route	Roles autorises	Description
POST	/api/v1/whatsapp/groups	Systeme (interne)	Creer un groupe de classe
POST	/api/v1/whatsapp/broadcast	Enseignant / Admin	Diffuser une annonce
GET	/api/v1/whatsapp/groups/{classId}	Enseignant	Consulter un groupe de classe

Communications inter-services

Service cible	Protocole / Evenement	Payload	Declencheur
-	Kafka (consumer) / student.enrolled	{classId, studentId}	Ajout automatique au groupe de classe

Cas d'utilisation associes

ID	Cas d'utilisation
UC19	Diffuser une annonce

S12 - pedagogic-service : Ressources Pedagogiques

Stack technique : Python FastAPI — Port 8089 — PostgreSQL, S3/MinIO

Description

Le pedagogic-service centralise la bibliotheque de ressources pedagogiques : documents, videos, plans de cours et outils d'evaluation destines aux enseignants. Les fichiers sont stockes sur MinIO/S3 et references en base PostgreSQL.

Responsabilites

- Upload et publication de ressources pedagogiques
- Gestion des cours et des plans de lecon
- Outils d'evaluation des enseignants

Endpoints REST

Methode	Route	Roles autorises	Description
POST	/api/v1/pedagogic/resources	Enseignant	Uploader une ressource
GET	/api/v1/pedagogic/resources	Enseignant	Lister les ressources disponibles
POST	/api/v1/pedagogic/courses	Enseignant	Creer un cours / plan de lecon

Communications inter-services

Service cible	Protocole / Evenement	Payload	Declencheur
-	-	-	Service faiblement couple, pas d'evenement emis

Cas d'utilisation associes

ID	Cas d'utilisation
-	Consulter / publier une ressource pedagogique

S13 - schoolid-service : Cartes d'Identite Scolaires

Stack technique : Python — Port 8090 — Pillow / ReportLab, PostgreSQL

Description

Le schoolid-service genere automatiquement les cartes d'identite scolaires avec QR code integre et photo, a la suite d'une inscription validee. Il gere egalement le renouvellement et la revocation des cartes.

Responsabilites

- Generation de cartes d'identite avec QR code
- Integration de la photo depuis l'inscription
- Impression en lot (bulk export PDF)
- Gestion de la validite et du renouvellement des cartes

Endpoints REST

Methode	Route	Roles autorises	Description
POST	/api/v1/school-id/generate	Systeme (interne)	Generer une carte ID
POST	/api/v1/school-id/{studentId}/reissue	Admin	Renouveler une carte ID
GET	/api/v1/school-id/{studentId}	Parent / Admin	Telecharger la carte ID

Communications inter-services

Service cible	Protocole / Evenement	Payload	Declencheur
notification-service	Kafka / schoolid.generated	{studentId, idCardUrl}	Carte generee -> notification

Cas d'utilisation associes

ID	Cas d'utilisation
UC7	Generer la carte d'identite scolaire

S14 - analytics-service : Tableaux de Bord et KPI

Stack technique : Python — Port 8091 — PostgreSQL/Oracle, Redis

Description

L'analytics-service agrege les donnees de tous les services metier (paiements, presence, notes) pour alimenter les tableaux de bord des directeurs et de l'administrateur systeme. Les requetes lourdes s'appuient sur des vues materialisees Oracle et un cache Redis (TTL 1h).

Responsabilites

- Calcul et agregation des indicateurs cles (KPI)
- Generation de tableaux de bord par etablissement
- Export de rapports (PDF / CSV)
- Mise en cache des tableaux de bord frequemment consultes

Endpoints REST

Methode	Route	Roles autorises	Description
GET	/api/v1/analytics/dashboard	Directeur / Admin	Tableau de bord d'un etablissement
GET	/api/v1/analytics/global	Admin Systeme	Statistiques multi-etablissement
GET	/api/v1/analytics/export	Admin	Exporter un rapport (PDF/CSV)

Communications inter-services

Service cible	Protocole / Evenement	Payload	Declencheur
-	Kafka (consumer multi-topics)	evenements paiement/presence/notes	Alimentation des indicateurs en continu

Cas d'utilisation associes

ID	Cas d'utilisation
UC22	Consulter le tableau de bord analytique

S15 - admin-service : Configuration et Administration

Stack technique : Java Spring Boot — Port 8092 — PostgreSQL

Description

L'admin-service permet a l'administrateur systeme de gerer les etablissements rattaches a la plateforme, la configuration technique globale et les comptes utilisateurs a l'echelle multi-etablissement.

Responsabilites

- Ajout / configuration des etablissements
- Gestion des comptes utilisateurs (activation, suspension)
- Parametrage global du systeme (tarifs, periodes scolaires)

Endpoints REST

Methode	Route	Roles autorises	Description
POST	/api/v1/admin/establishments	Admin Systeme	Ajouter un etablissement
PATCH	/api/v1/admin/users/{id}/status	Admin Systeme	Activer / suspendre un compte
GET	/api/v1/admin/config	Admin Systeme	Consulter la configuration globale

Communications inter-services

Service cible	Protocole / Evenement	Payload	Declencheur
auth-service	REST / HTTP	{userId, status}	Suspension d'un compte utilisateur

Cas d'utilisation associés

ID	Cas d'utilisation
UC23	Gerer la configuration multi-etablissement

6. Architecture Logicielle

6.1 Vue Globale de l'Architecture Micro-services

La plateforme SchoolManage App est construite selon une architecture micro-services. Chaque service est independant, dispose de sa propre base de donnees (pattern Database per Service) et communique avec les autres services soit de facon synchrone (REST/HTTP via l'API Gateway), soit de facon asynchrone (evenements Kafka/RabbitMQ).

Composants structurels principaux :

- API Gateway (Spring Cloud Gateway / Kong) : point d'entree unique, validation JWT, routage
- auth-service : gardien central RBAC/JWT pour toutes les requetes
- 12 micro-services metier independants avec bases PostgreSQL dediees
- Bus de messages Kafka/RabbitMQ pour les communications asynchrones inter-services

Architecture Generale - SchoolManage App

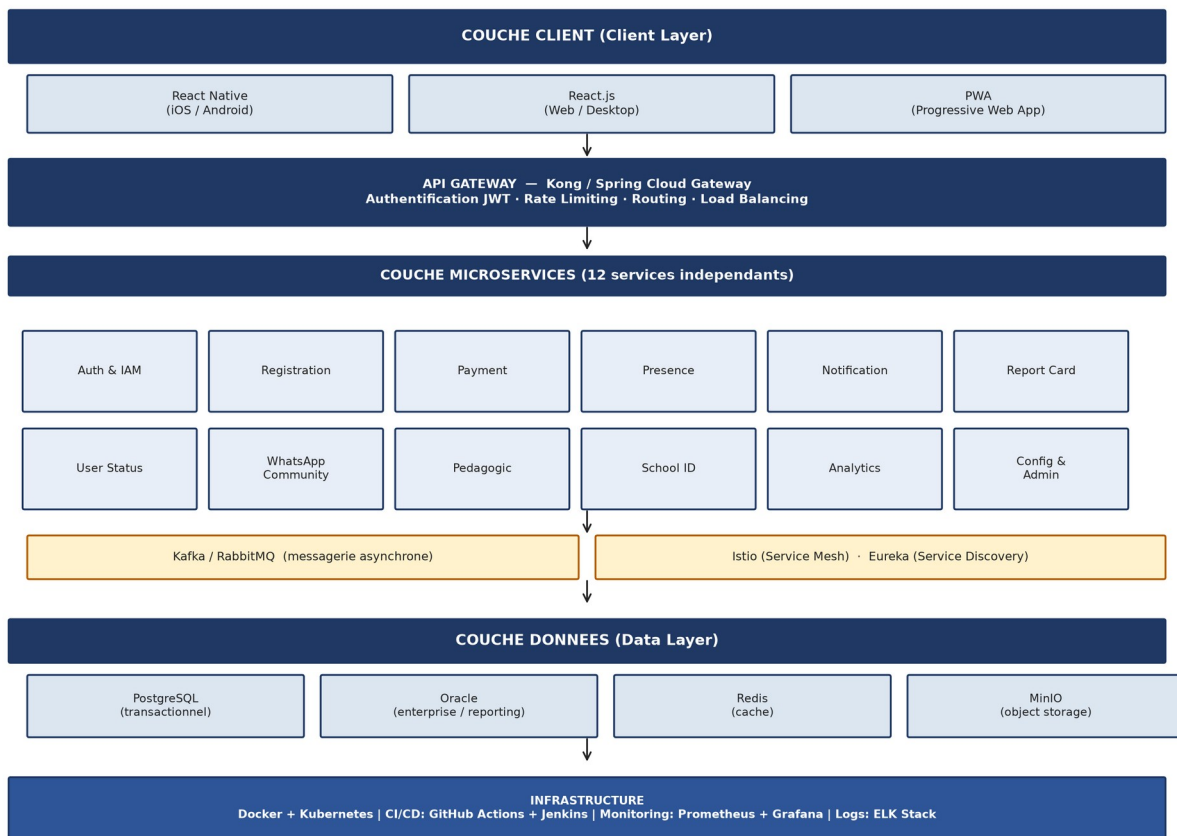


Figure 2 - Architecture generale de la plateforme

6.2 Communications Inter-services - Vue Globale

Le tableau ci-dessous synthetise l'ensemble des communications entre les 12 micro-services metier (Kafka/RabbitMQ, REST/HTTP et webhooks).

Service source	Service cible	Protocole	Objet de la communication
auth-service	registration-service	Kafka	Creation de profil initial a l'inscription
auth-service	notification-service	Kafka	Email de bienvenue / reinitialisation mot de passe
registration-service	schoolid-service	Kafka	Inscription validee -> generation carte ID
registration-service	whatsapp-service	Kafka	Ajout automatique au groupe de classe
payment-service	notification-service	Kafka	Envoi du reçu / notification d'echec
payment-service	analytics-service	Kafka	Mise a jour des indicateurs financiers
presence-service	notification-service	Kafka	Alerte d'absence au parent
reportcard-service	notification-service	Kafka	Bulletin disponible -> notification
userstatus-service	notification-service	Kafka	Changement de statut -> notification
admin-service	auth-service	REST/HTTP	Suspension / activation de compte
MTN / Orange Money	payment-service	HTTPS Webhook	Confirmation ou refus de paiement
API Gateway	auth-service	REST/HTTP	Validation JWT pour chaque requete entrante

6.3 Strategies Techniques Cles

Gestion de la coherence des donnees

- Pattern Saga (choreographie) via Kafka pour les flux d'inscription et de paiement
- Verrou distribue Redis pour la prevention des doublons de check-in
- Cles d'idempotence Redis pour la prevention des doubles debits
- Vues materialisees Oracle pour les rapports sans impact transactionnel

Performance et scalabilite

- Cache Redis adaptatif par service (TTL 5 min recherche, TTL 1h dashboards)
- Base PostgreSQL dediee par service (isolation totale des donnees)
- Read replicas PostgreSQL pour les services a fort volume de lecture
- Kafka avec messages persistants pour garantir la livraison des evenements

Securite

- JWT (access 15 min + refresh 7 jours) avec blacklist Redis
- RBAC : 4 roles (Parent, Enseignant, Administrateur, Directeur) avec permissions dediees
- HTTPS obligatoire sur tous les endpoints publics et webhooks
- Validation et sanitisation des entrees (OWASP Top 10)

7. Diagrammes de Conception

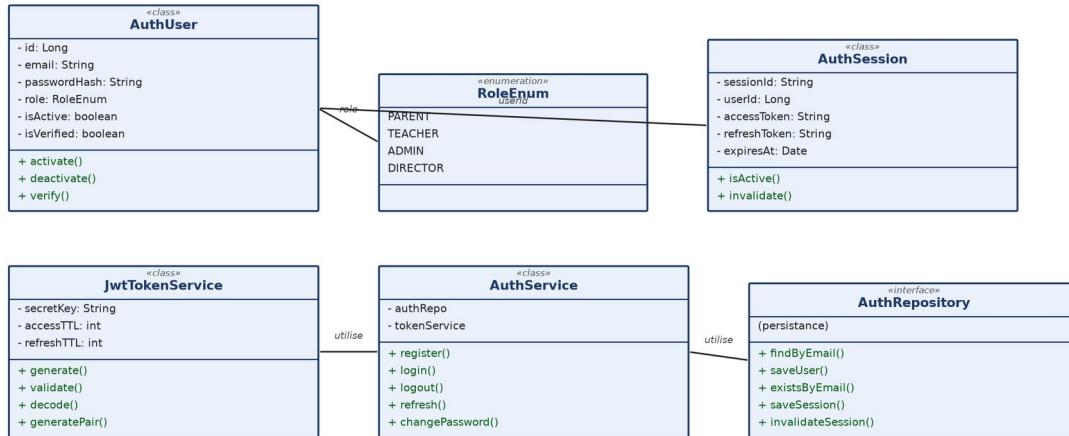
Cette section presente l'ensemble des diagrammes UML et techniques realises pour la plateforme SchoolManage App : diagrammes de classes, de sequence, d'activite, d'etat-transition et de composants, par micro-service.

7.1 Diagrammes de Classes

Les diagrammes de classes decrivent la structure statique des entites de chaque micro-service.

a. Auth & IAM Service

Diagramme de Classes - Auth & IAM Service



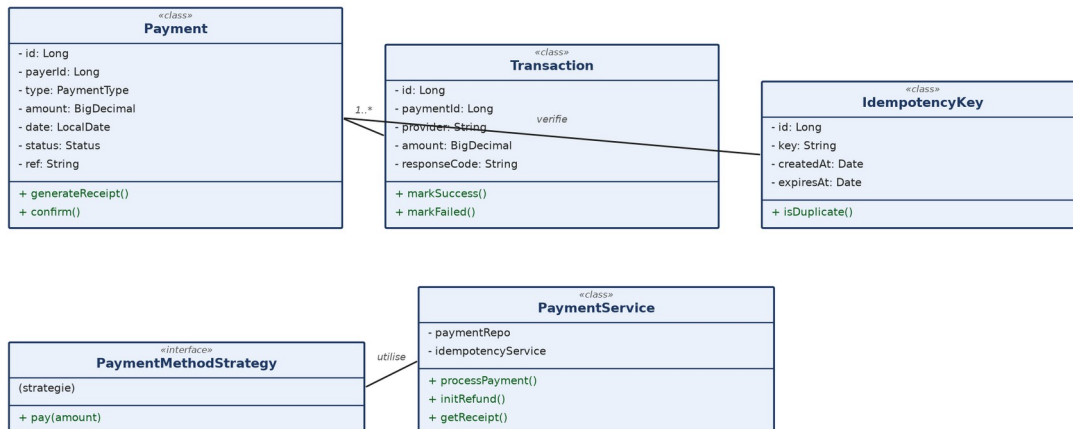
b. Registration & School ID Service

Diagramme de Classes - Registration & School ID Service



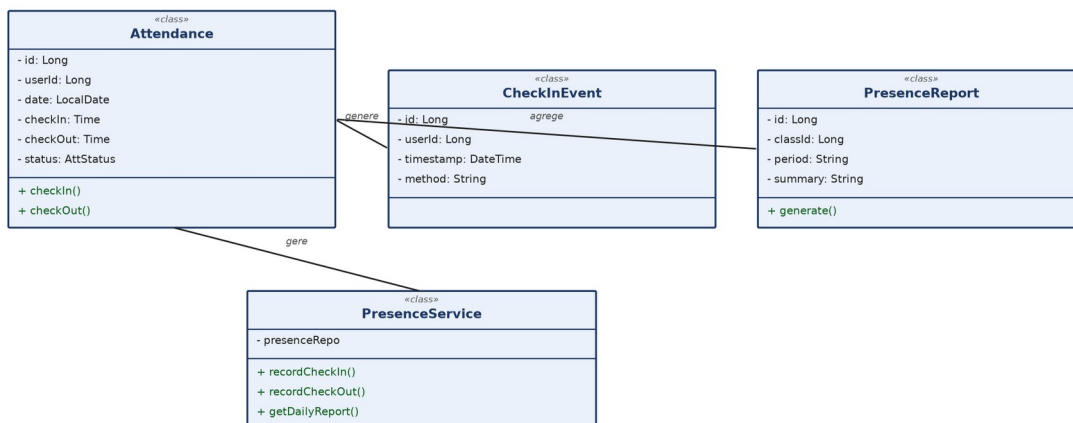
c. Payment Service

Diagramme de Classes - Payment Service



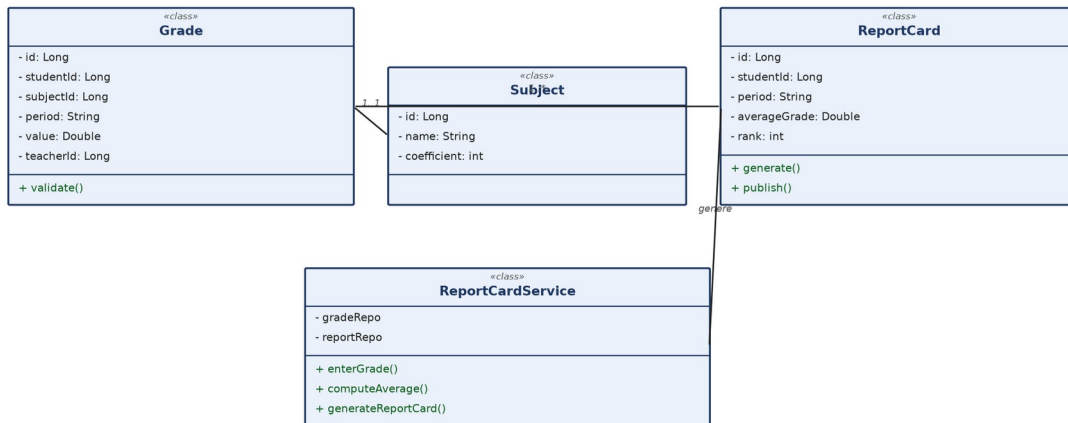
d. Presence Service

Diagramme de Classes - Presence Service



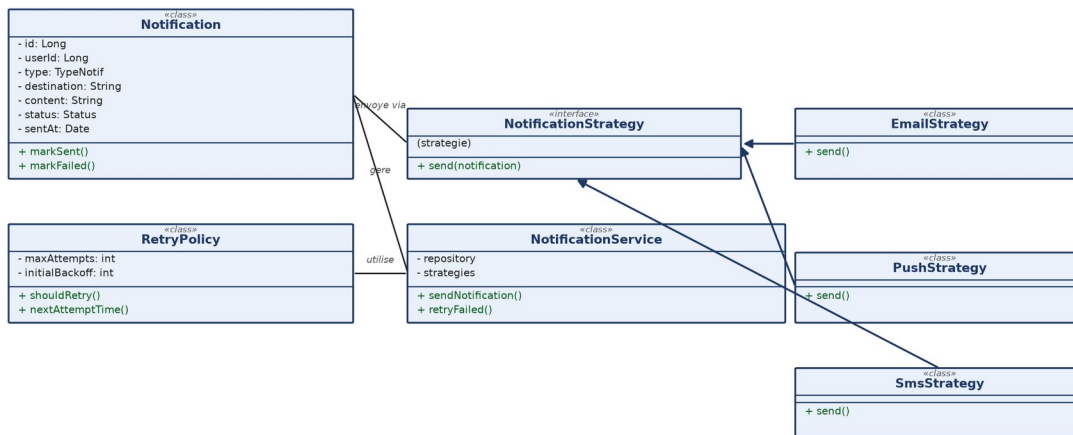
e. Report Card Service

Diagramme de Classes - Report Card Service



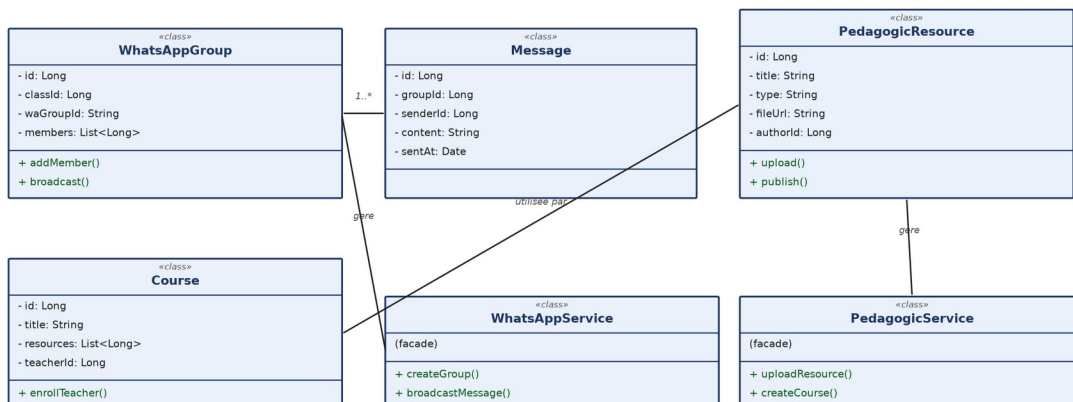
f. Notification Service

Diagramme de Classes - Notification Service



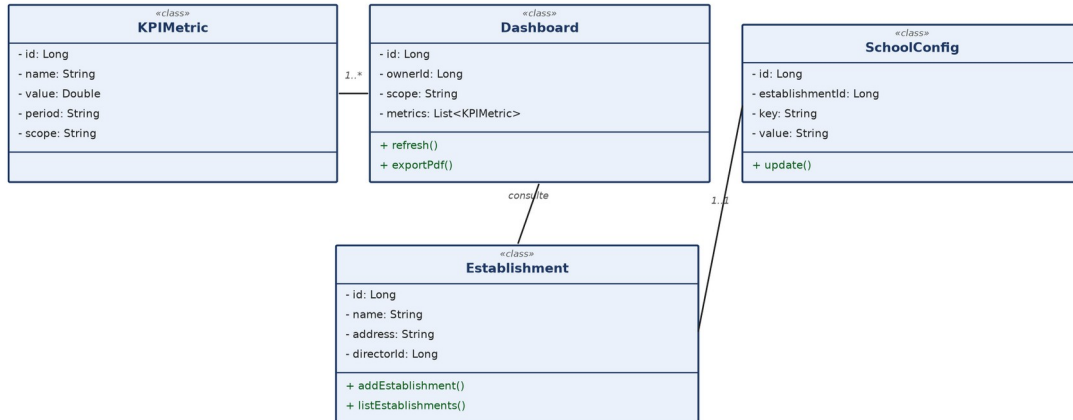
g. WhatsApp Community & Pedagogic Service

Diagramme de Classes - WhatsApp Community & Pedagogic Service



h. Analytics & Config/Admin Service

Diagramme de Classes - Analytics & Config/Admin Service

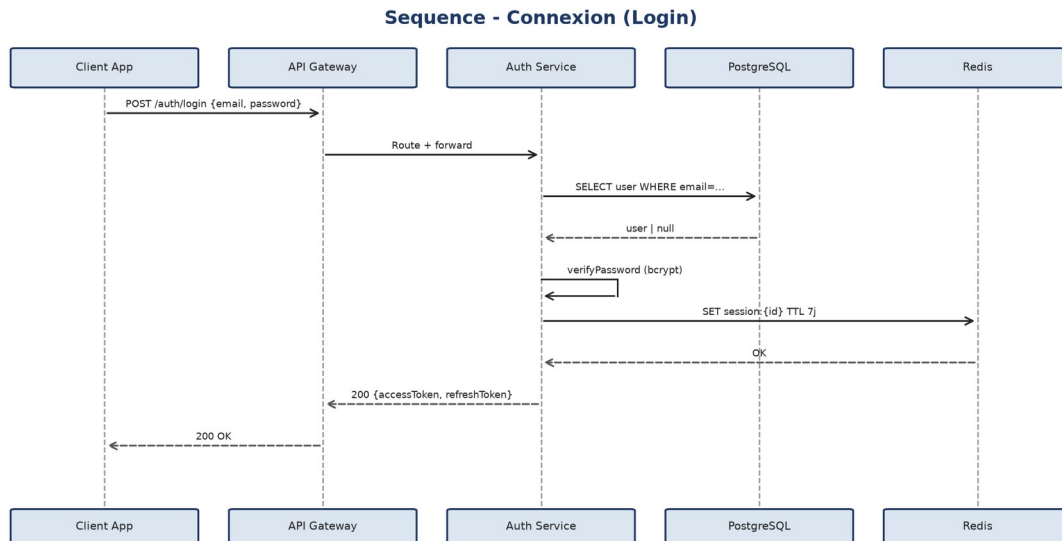


7.2 Diagrammes de Sequence par Cas d'Utilisation

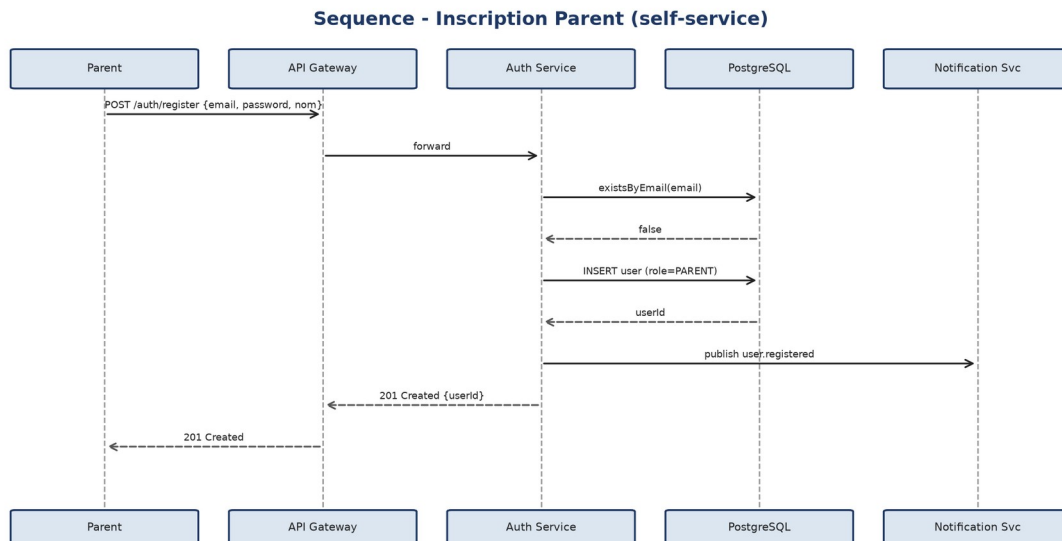
Les diagrammes de sequence illustrent les interactions dynamiques entre acteurs et services pour les flux principaux.

7.2.1 Auth & IAM Service

• Login

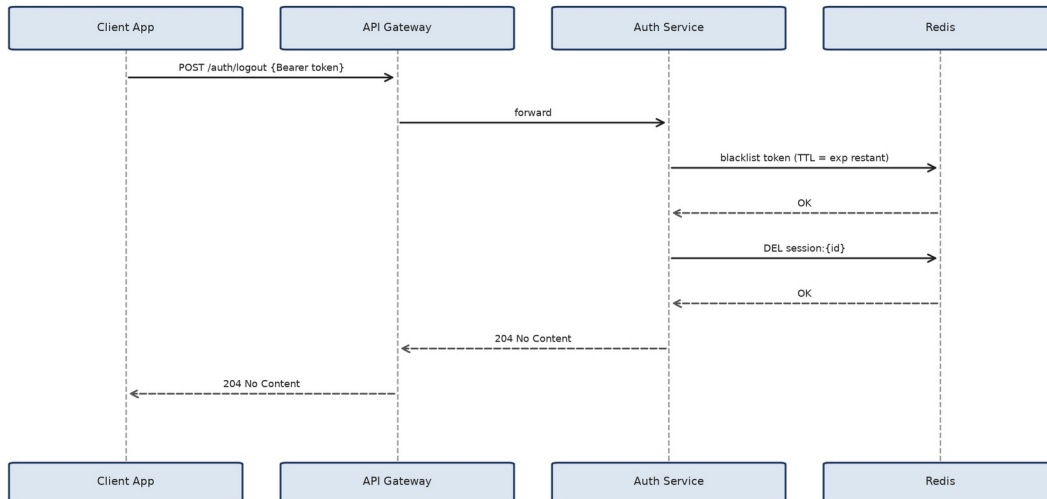


• Register (self-service)



• Logout

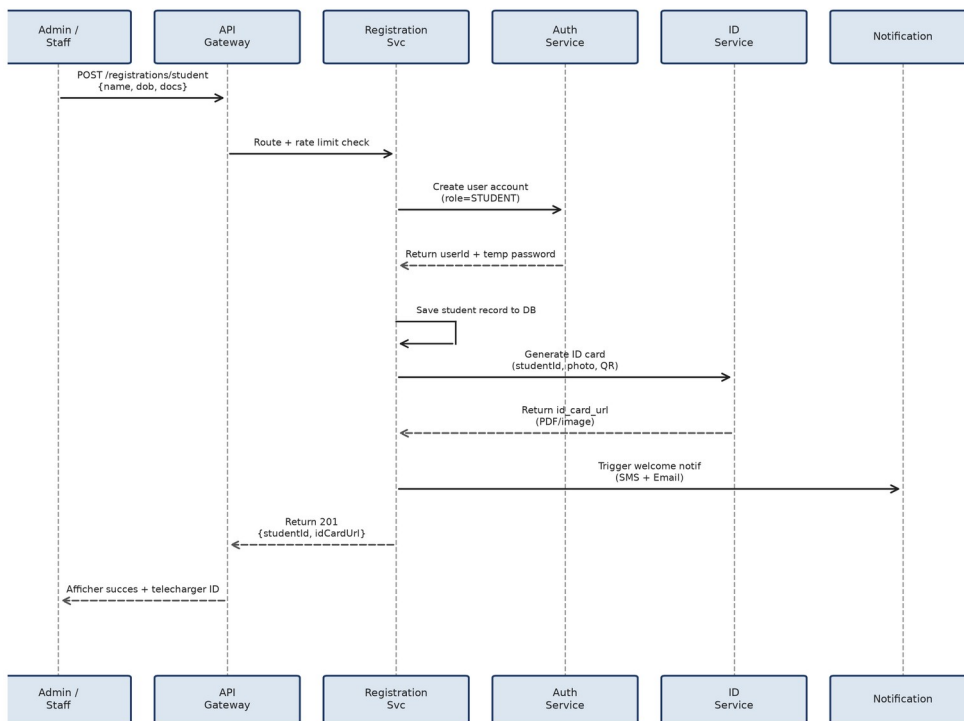
Sequence - Deconnexion (Logout)



7.2.2 Registration & School ID Service

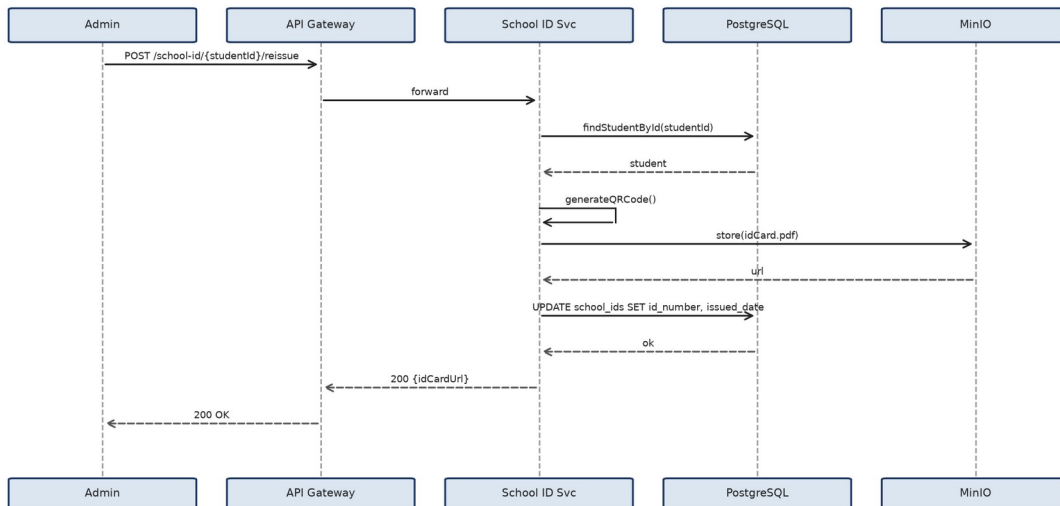
• Inscription et generation de carte ID

Diagramme de Sequence - Inscription & Generation d'ID



• Renouveler la carte d'identite scolaire

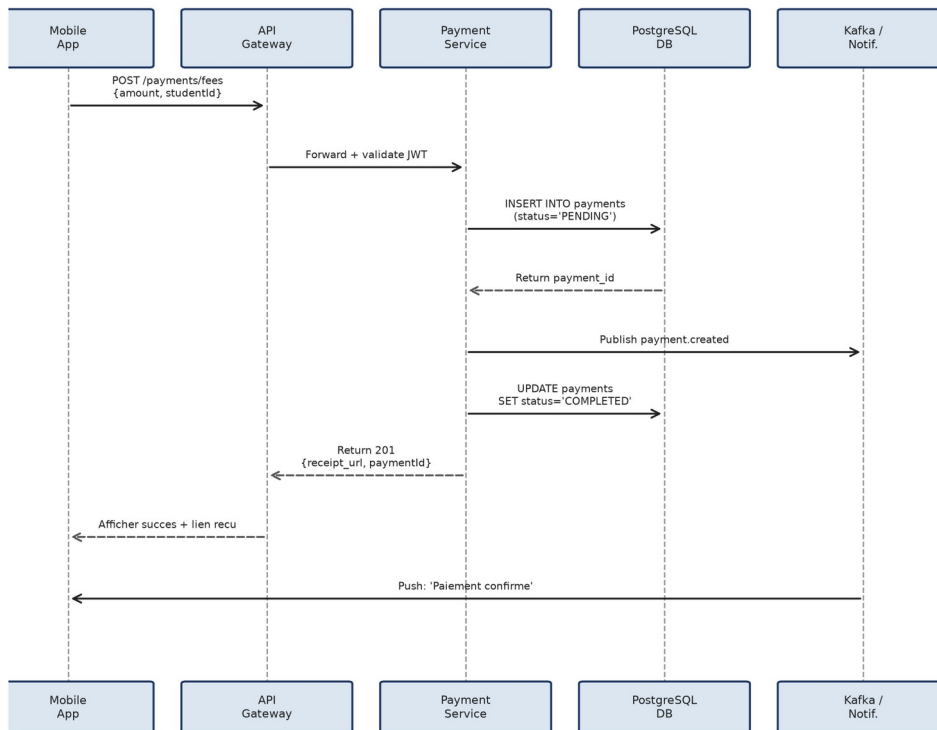
Sequence - Renouveler la carte d'identité scolaire



7.2.3 Payment Service

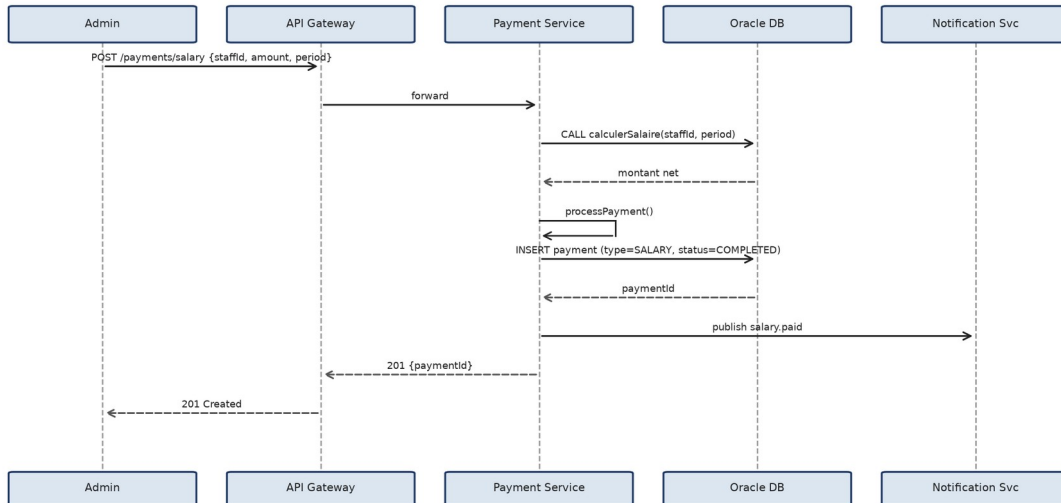
• Flux de paiement des frais

Diagramme de Sequence - Flux de Paiement



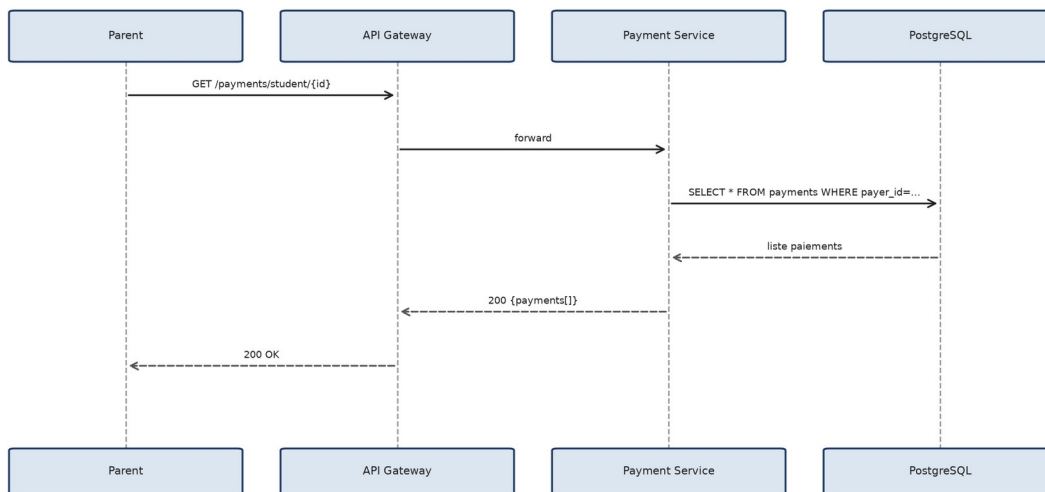
• Payer un salaire

Sequence - Payer un salaire (staff)



• Consulter l'historique des paiements

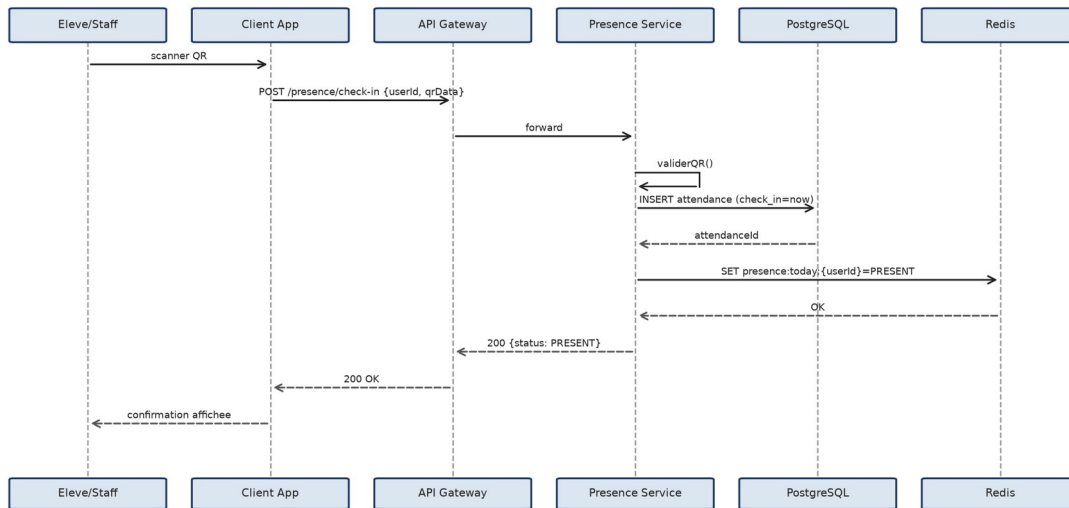
Sequence - Consulter l'historique des paiements



7.2.4 Presence Service

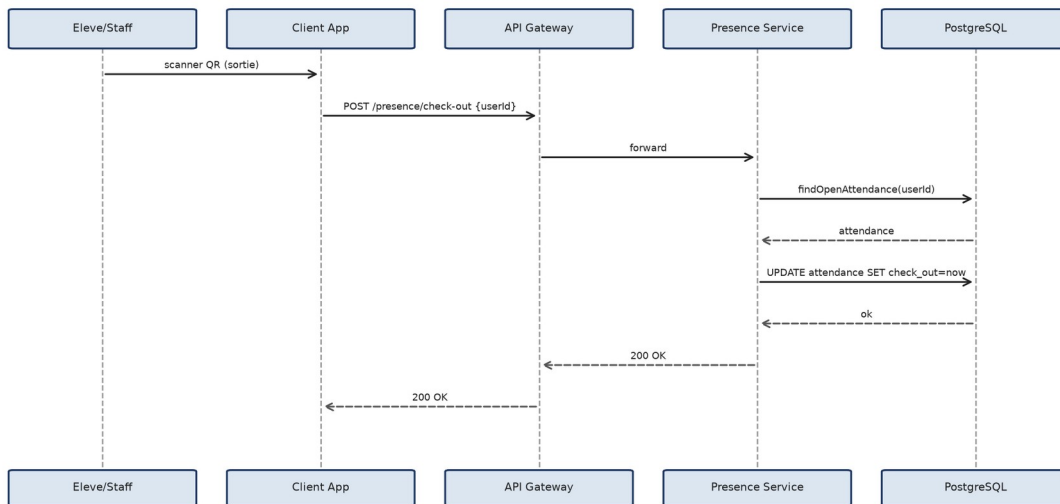
• Check-in par QR code

Sequence - Check-in par QR code



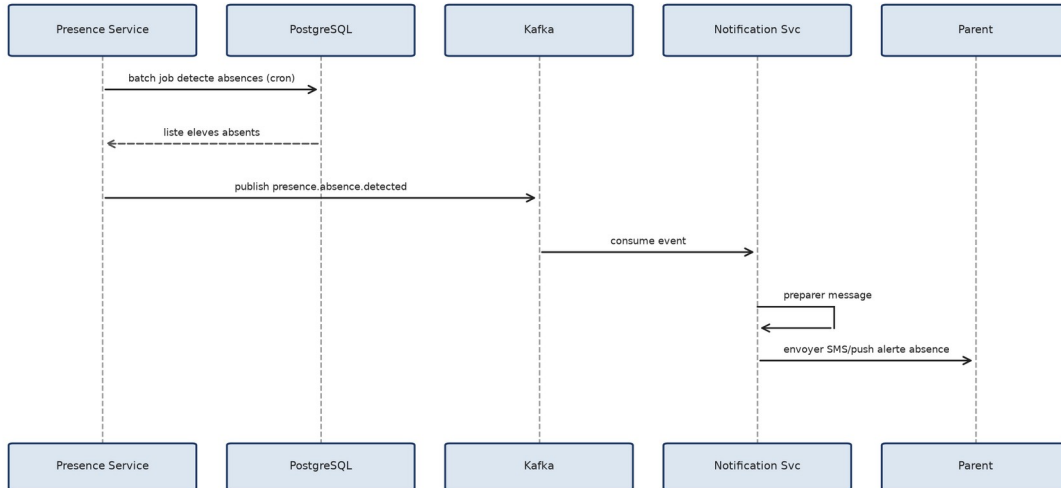
• Check-out par QR code

Sequence - Check-out par QR code



• Alerte d'absence automatique

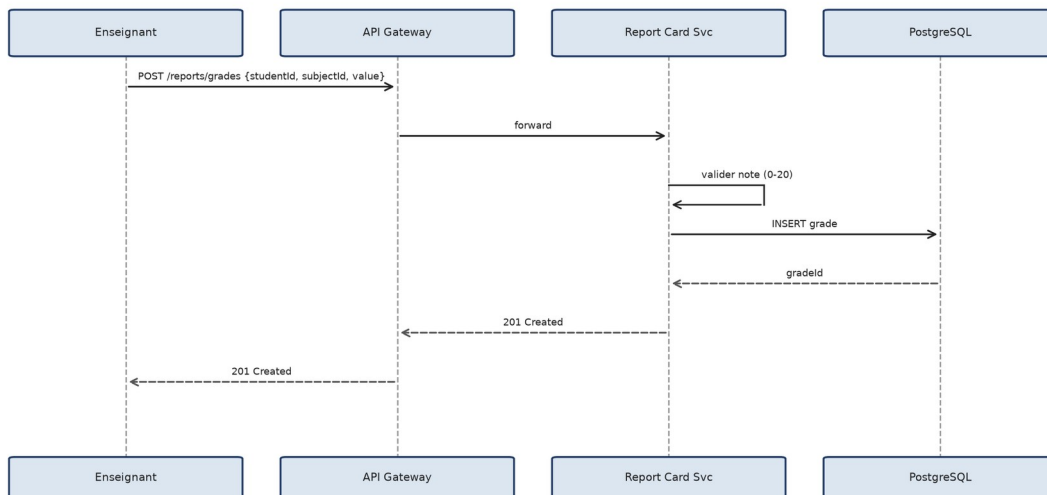
Sequence - Alerte d'absence automatique



7.2.5 Report Card Service

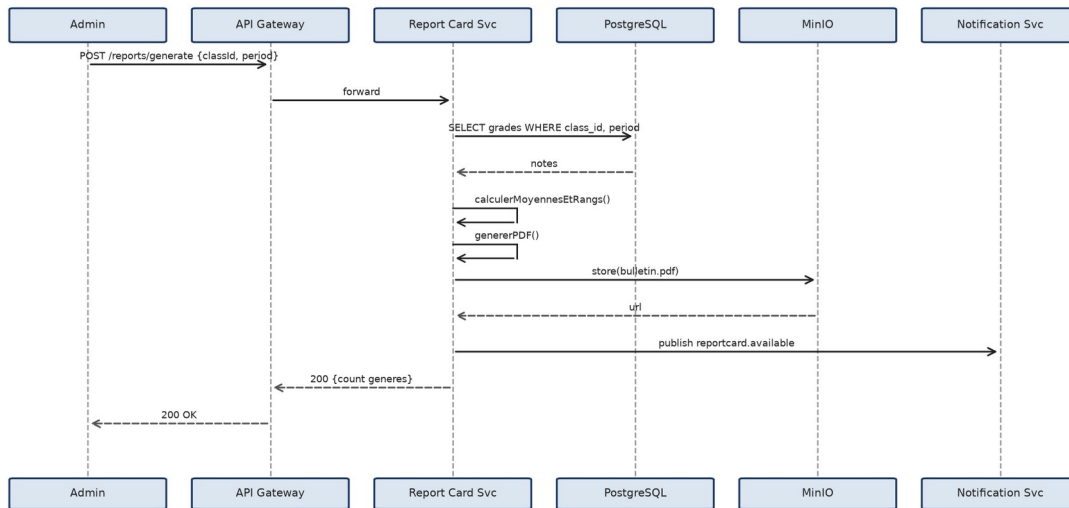
• Saisir les notes

Sequence - Saisir les notes



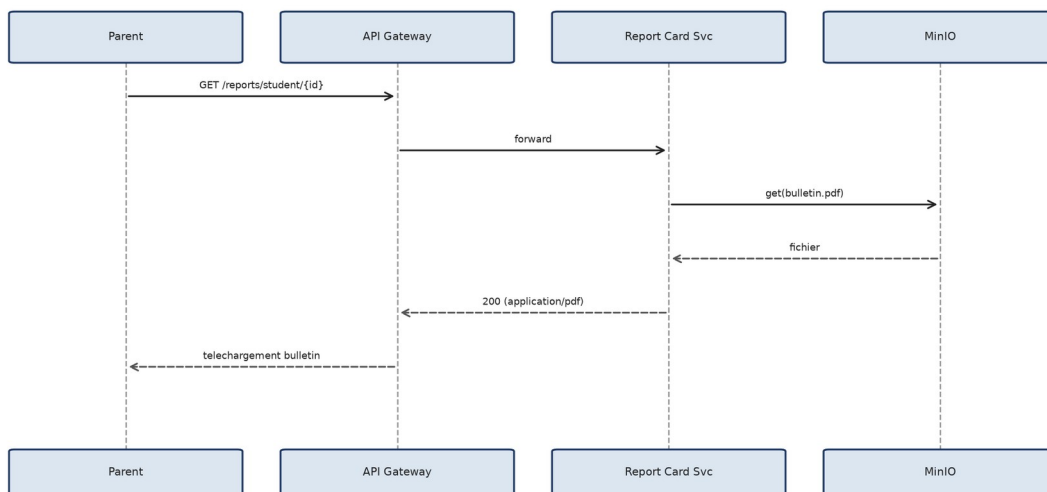
• Generer les bulletins

Sequence - Generer les bulletins



• Consulter le bulletin (parent)

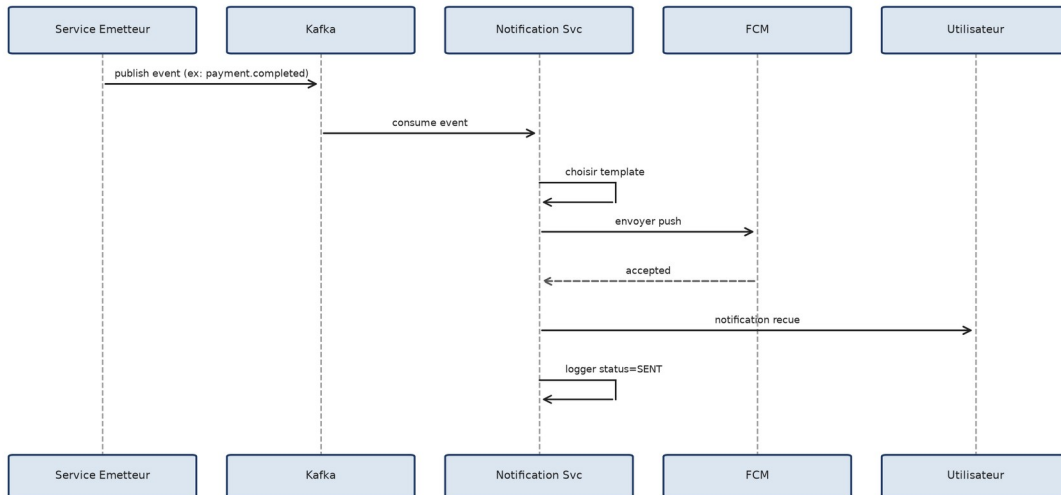
Sequence - Consulter le bulletin (parent)



7.2.6 Notification Service

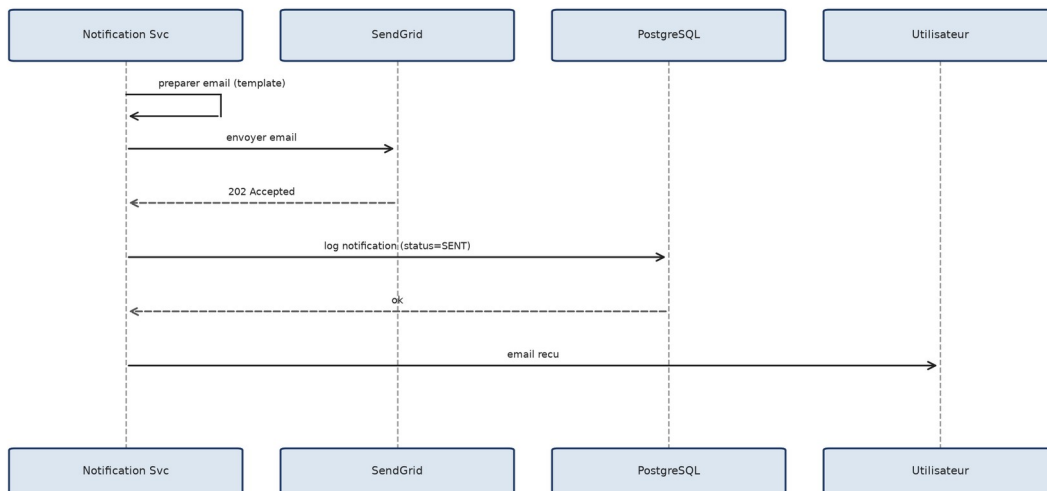
• Envoi d'une notification Push

Sequence - Envoi d'une notification Push



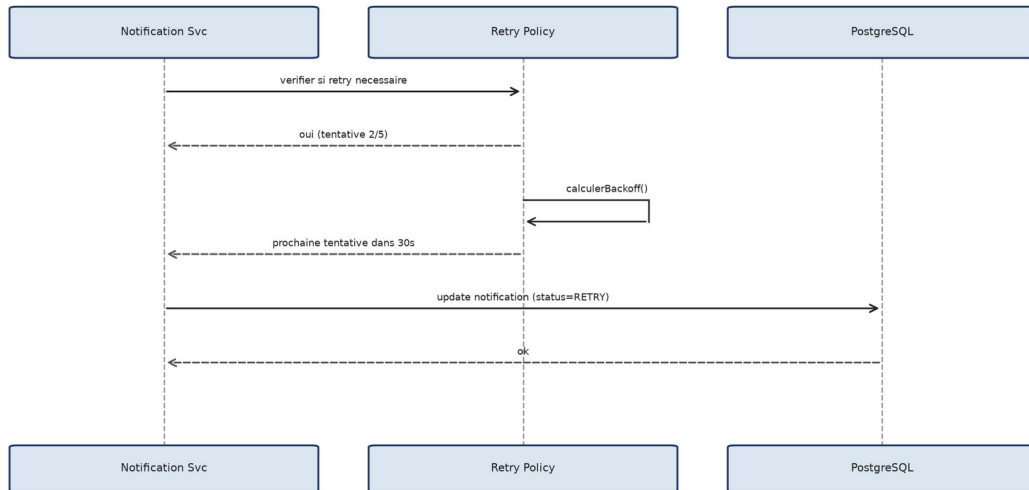
• Envoi d'une notification Email

Sequence - Envoi d'une notification Email



• Gestion des essais (retry)

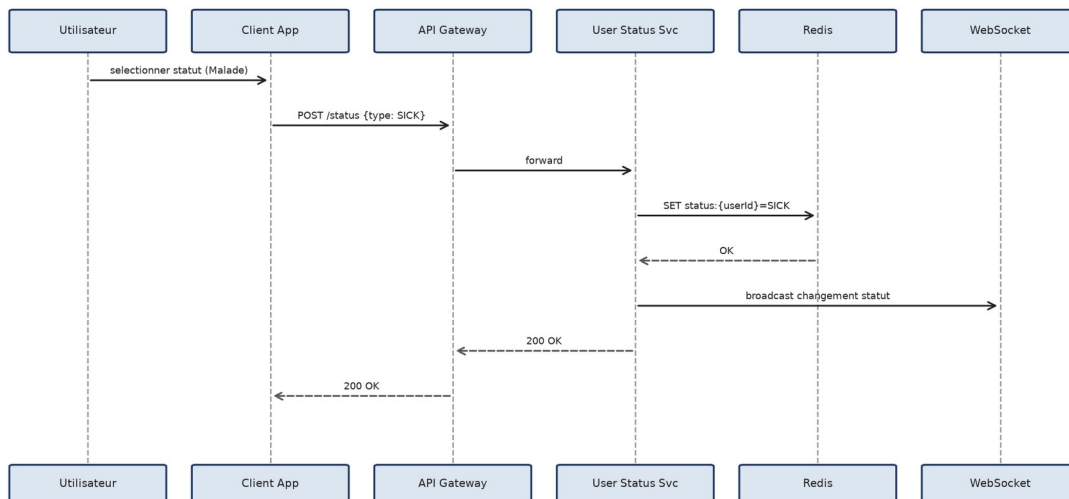
Sequence - Gestion des essais (retry)



7.2.7 User Status Service

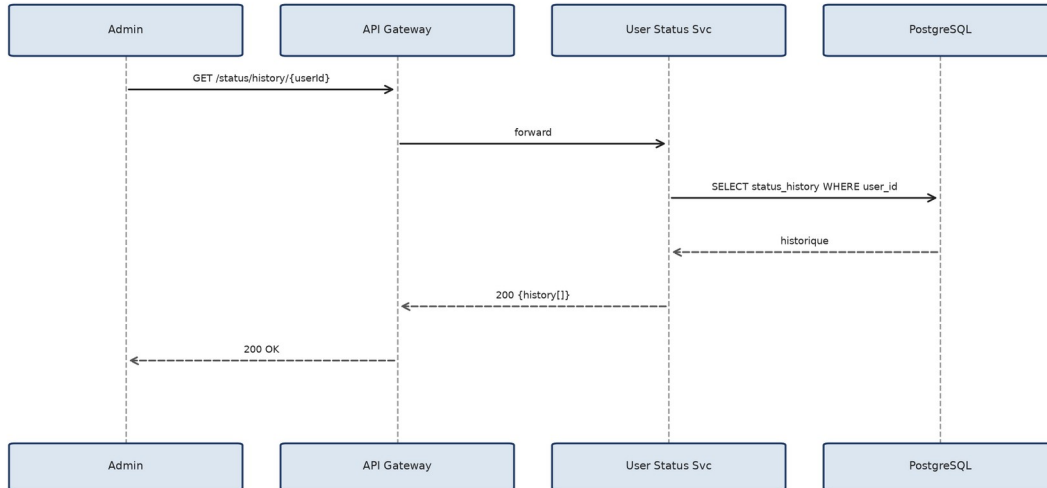
• Publier un statut utilisateur

Sequence - Publier un statut utilisateur



• Consulter l'historique de statut

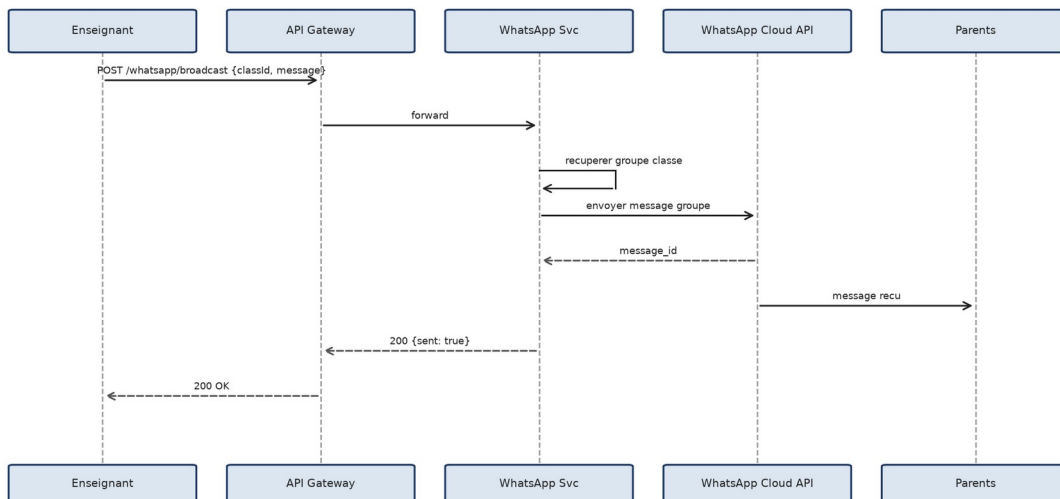
Sequence - Consulter l'historique de statut



7.2.8 WhatsApp Community Service

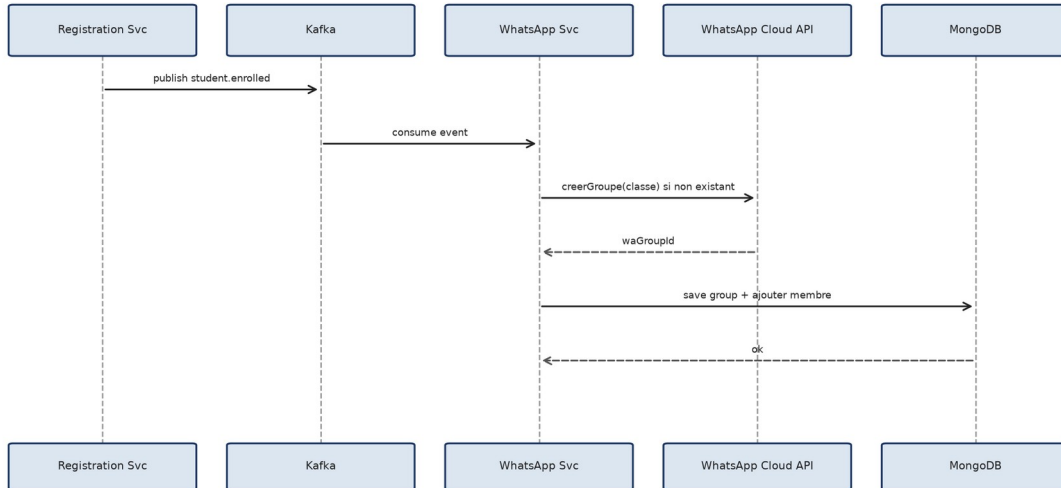
• Diffuser une annonce WhatsApp

Sequence - Diffuser une annonce WhatsApp



• Creation automatique d'un groupe de classe

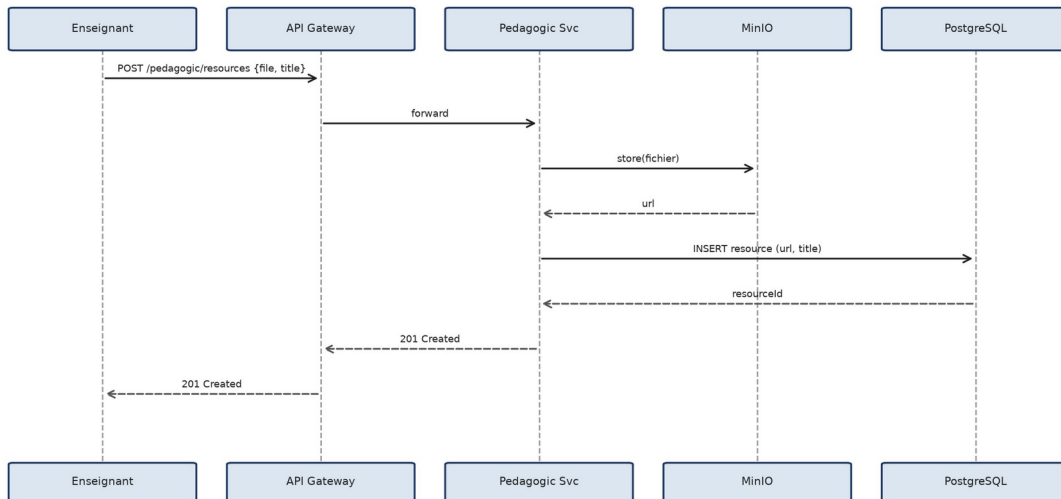
Sequence - Creation automatique d'un groupe de classe



7.2.9 Pedagogic Service

• Uploader une ressource pedagogique

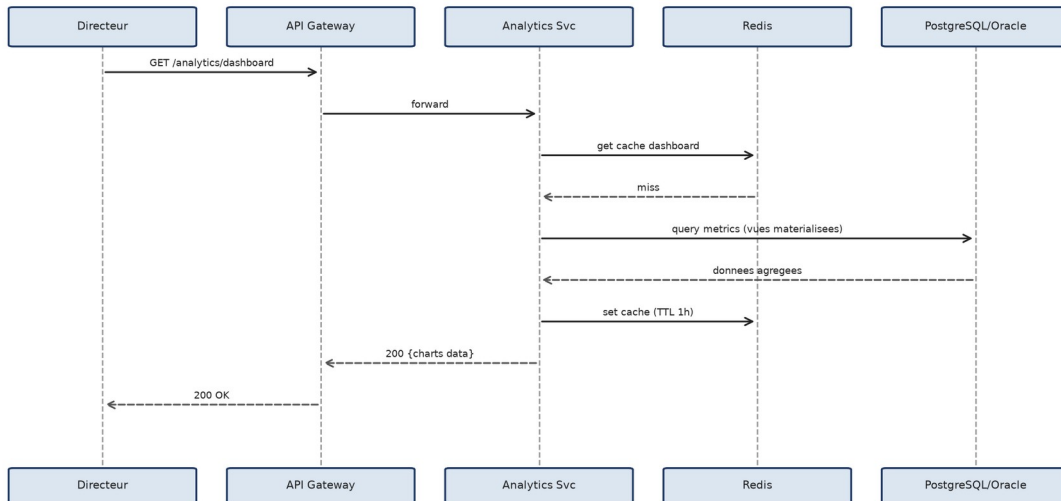
Sequence - Uploader une ressource pedagogique



7.2.10 Analytics Service

• Consulter le tableau de bord analytique

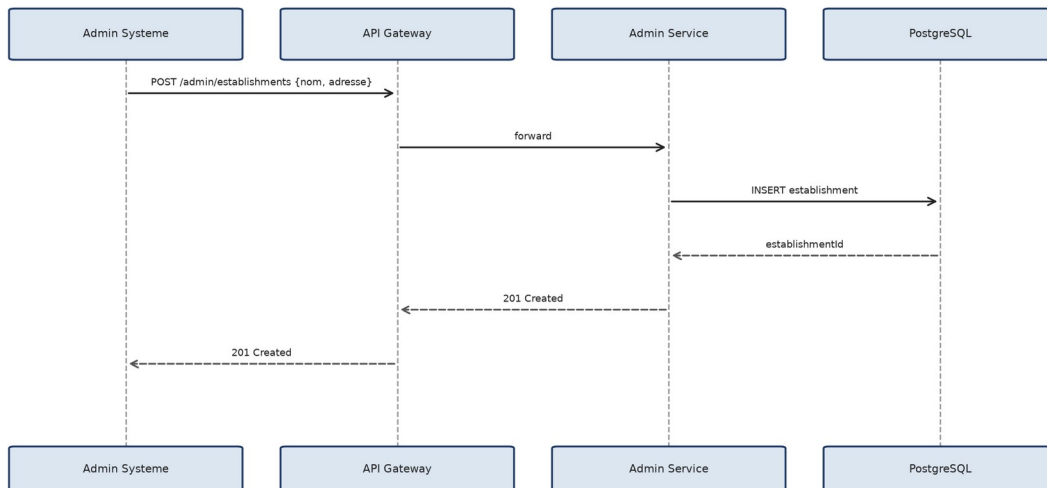
Sequence - Consulter le tableau de bord analytique



7.2.11 Admin Service

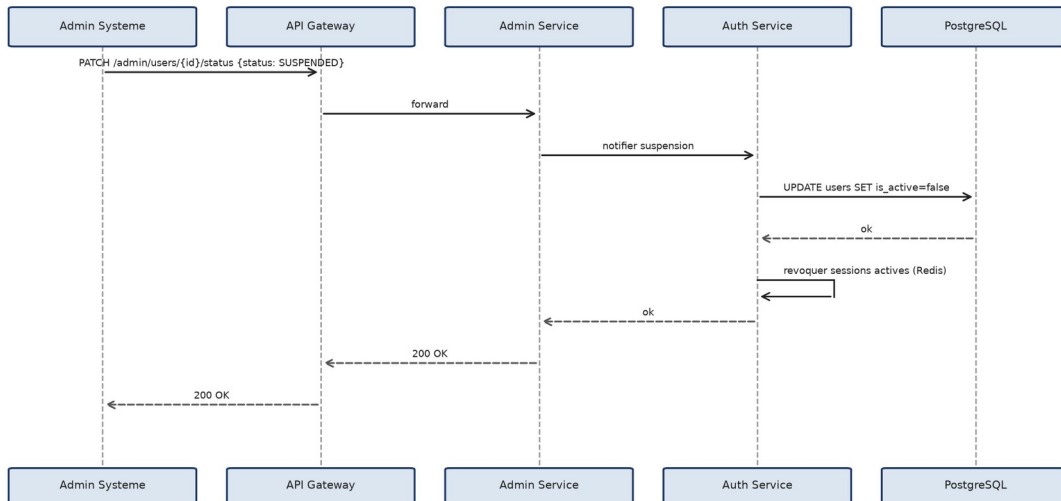
• Configurer un nouvel etablissement

Sequence - Configurer un nouvel etablissement



• Desactiver un compte utilisateur

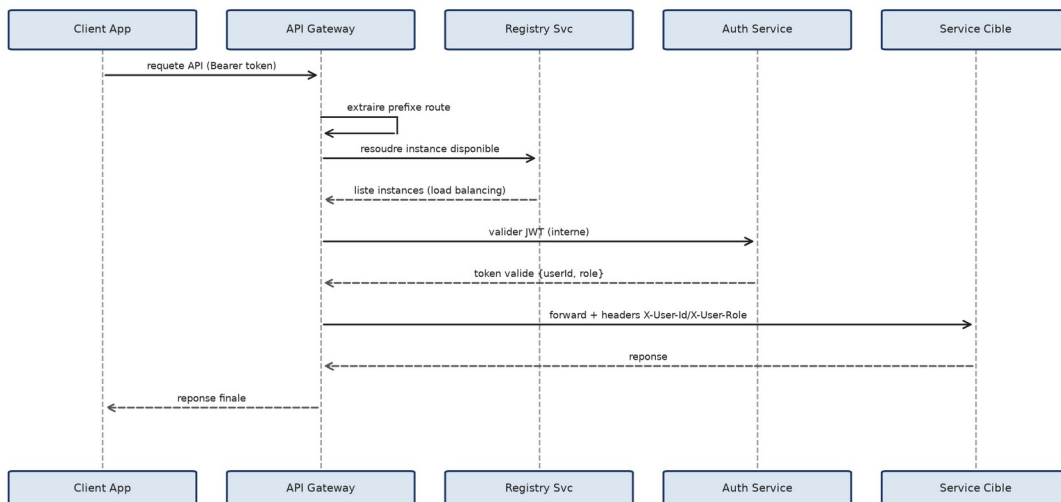
Sequence - Desactiver un compte utilisateur



7.2.12 Infrastructure (Gateway)

• Routage et validation JWT via le Gateway

Sequence - Routage et validation JWT via le Gateway

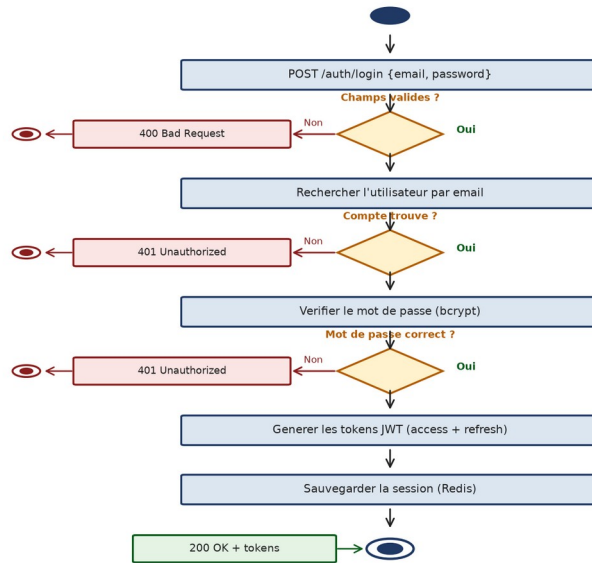


7.3 Diagrammes d'Activite

Les diagrammes d'activite modelisent les flux de traitement metier, incluant les decisions et les cas d'erreur.

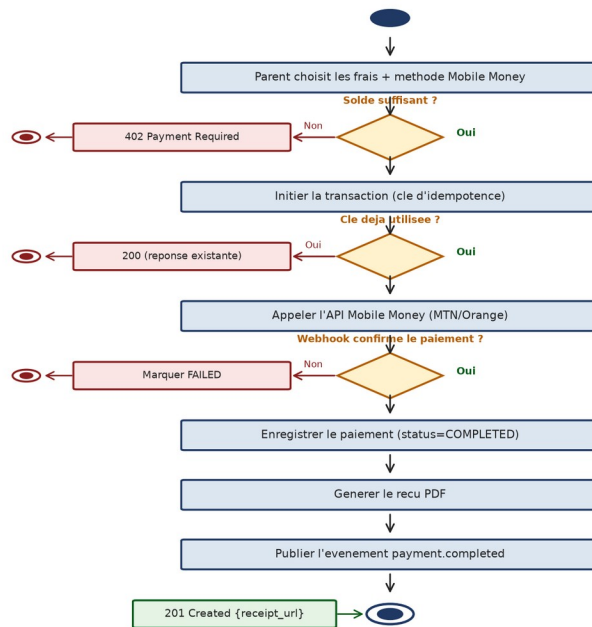
7.3.1 Authentification - Login

Diagramme d'Activite - Authentification (Login)



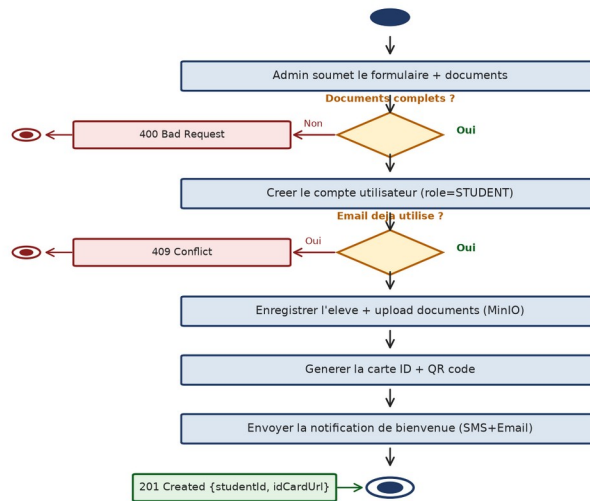
7.3.2 Paiement des frais de scolarite

Diagramme d'Activite - Paiement des Frais de Scolarite



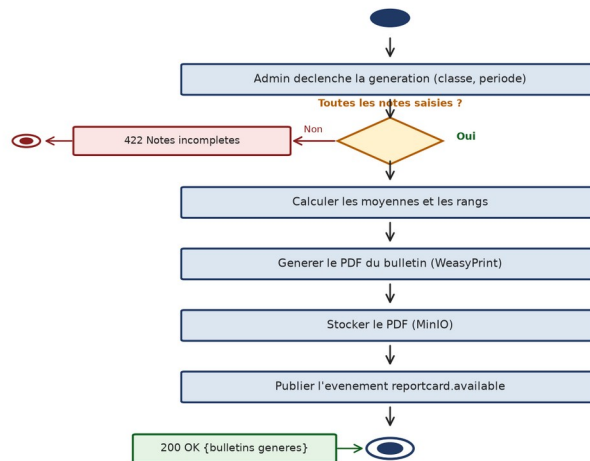
7.3.3 Inscription et generation d'ID

Diagramme d'Activite - Inscription & Generation d'ID



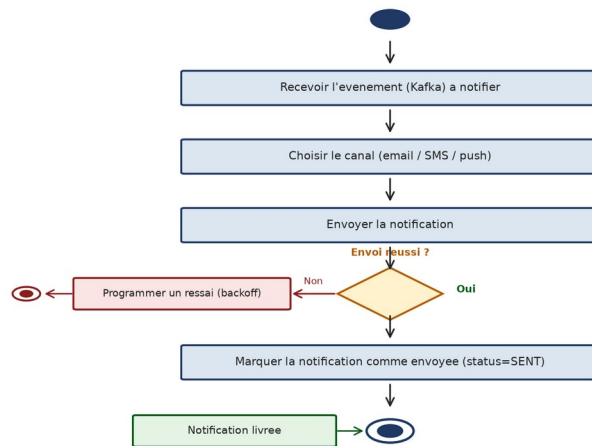
7.3.4 Generation des bulletins

Diagramme d'Activite - Generation des Bulletins



7.3.5 Envoi de notification avec resai

Diagramme d'Activite - Envoi de Notification avec Resai

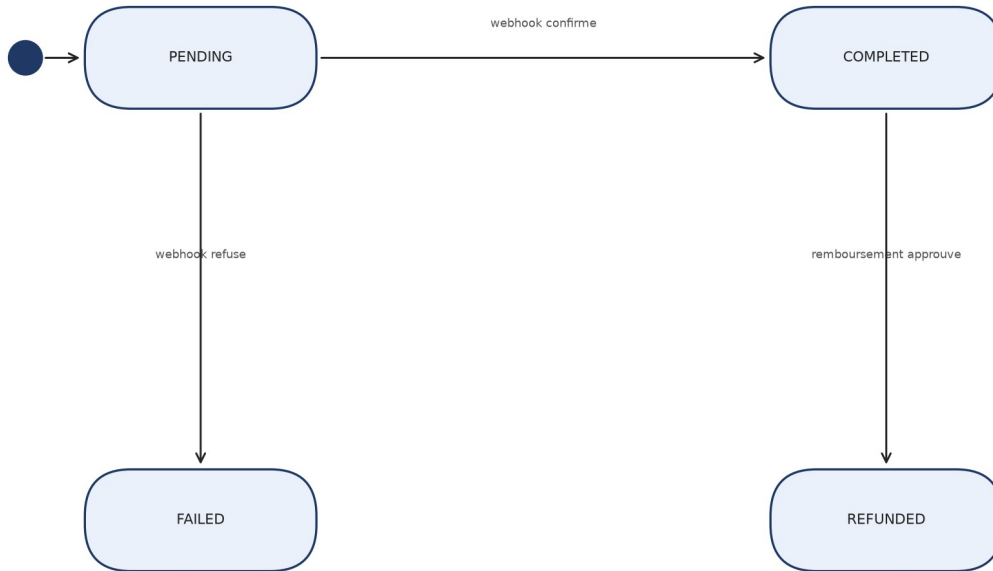


7.4 Diagrammes d'Etat-Transition

Ces diagrammes représentent le cycle de vie des entités métier les plus critiques.

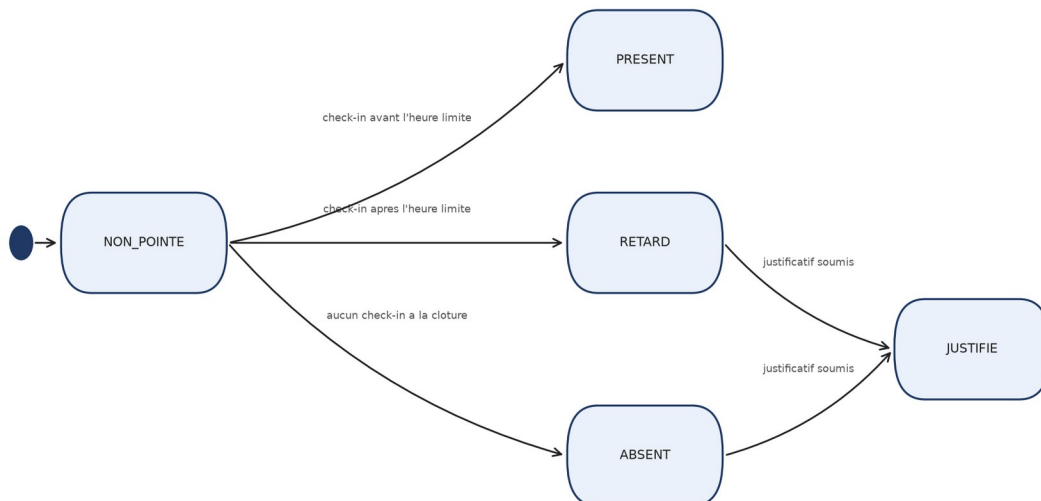
7.4.1 Etats d'un Paiement

Diagramme d'Etat-Transition - Paiement



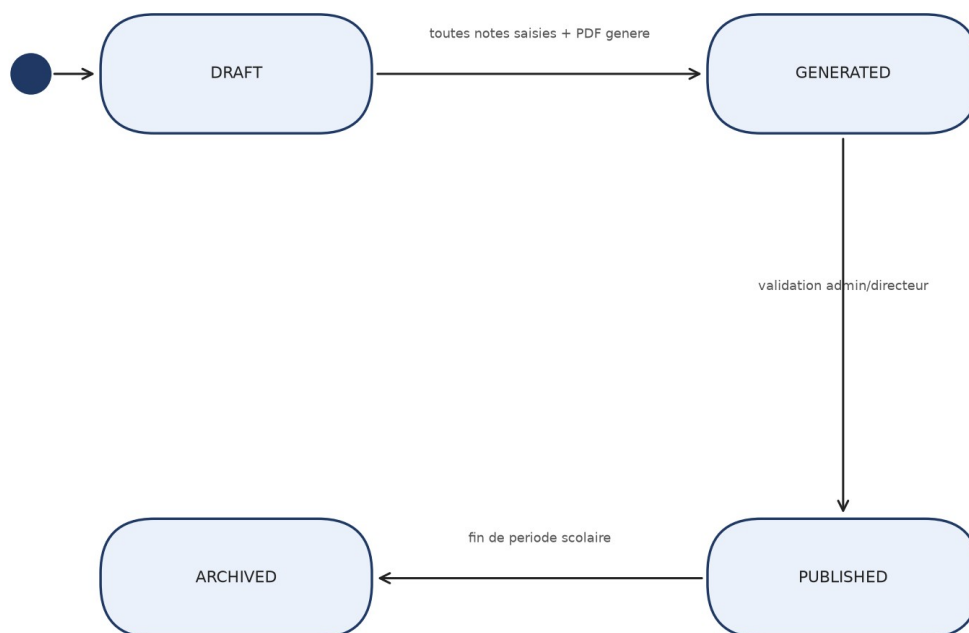
7.4.2 Etats d'une Presence (Check-in)

Diagramme d'Etat-Transition - Presence (Check-in)



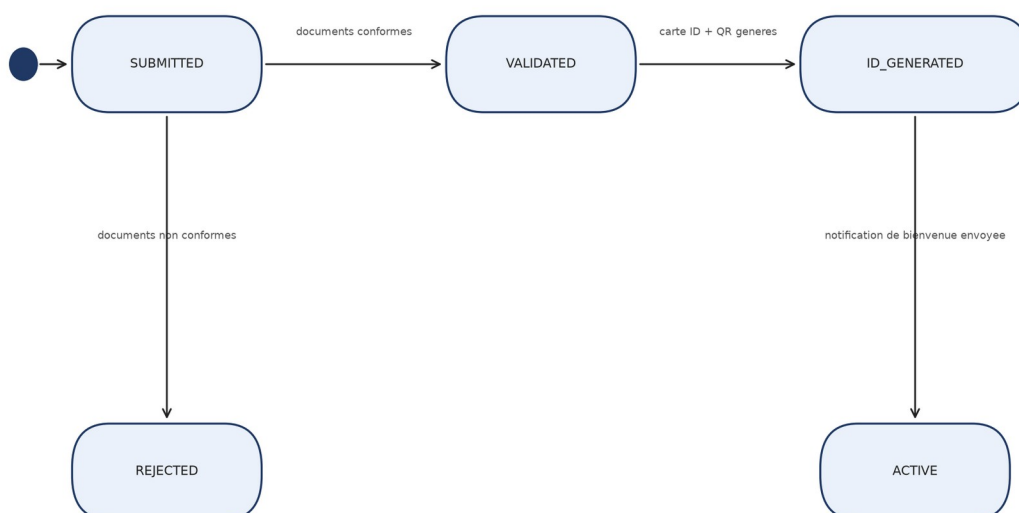
7.4.3 Etats d'un Bulletin

Diagramme d'Etat-Transition - Bulletin



7.4.4 Etats d'une Inscription

Diagramme d'Etat-Transition - Inscription

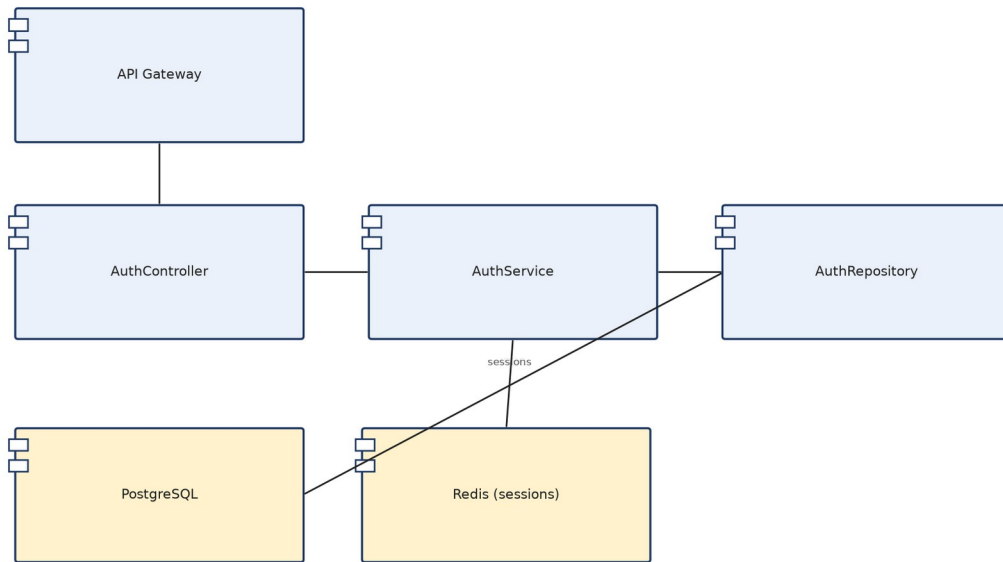


7.5 Diagrammes de Composants

Les diagrammes de composants montrent l'organisation interne (contrôleur, service, dépôt) et les dépendances externes de chaque micro-service.

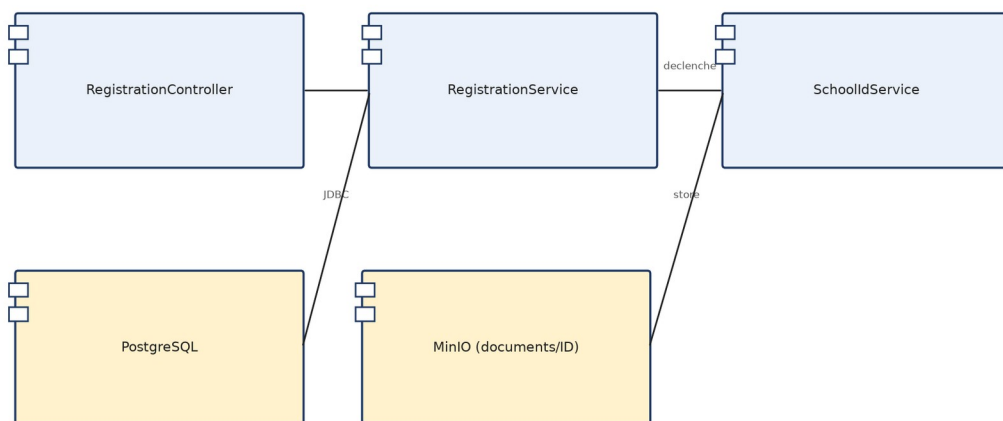
7.5.1 Auth & IAM Service

Diagramme de Composants - Auth & IAM Service



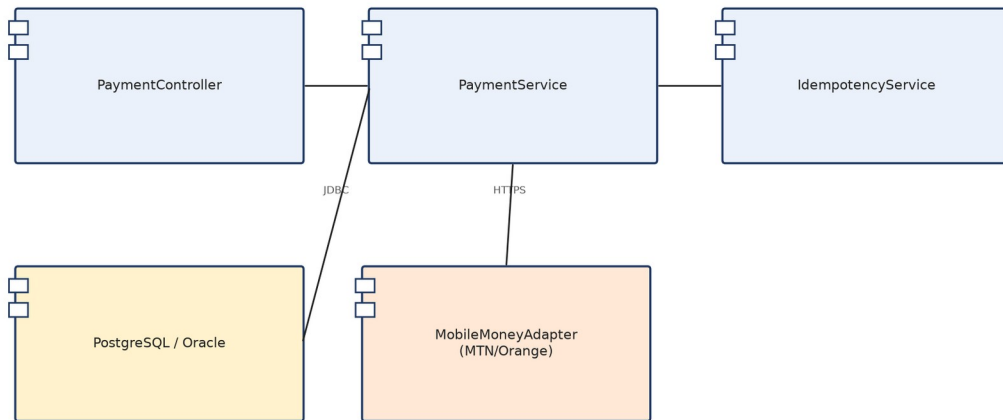
7.5.2 Registration & School ID Service

Diagramme de Composants - Registration & School ID Service



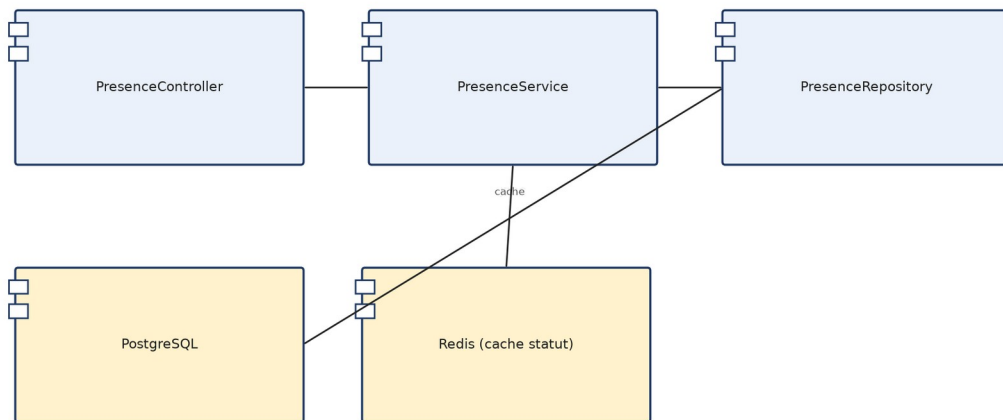
7.5.3 Payment Service

Diagramme de Composants - Payment Service



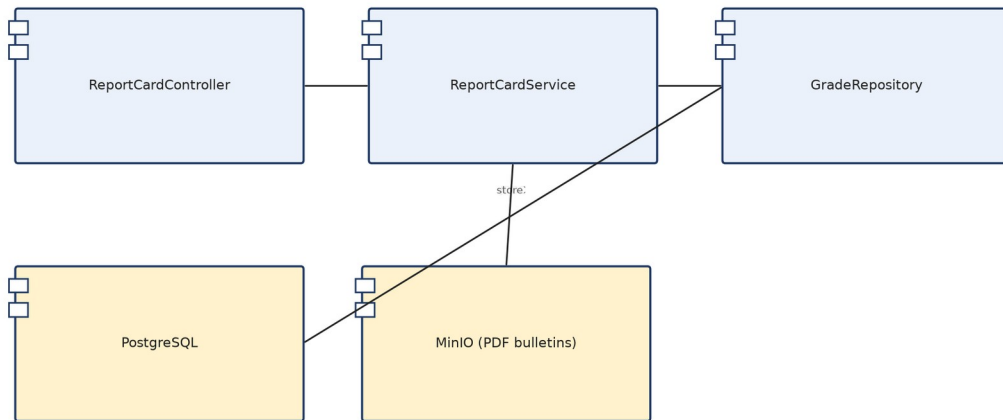
7.5.4 Presence Service

Diagramme de Composants - Presence Service



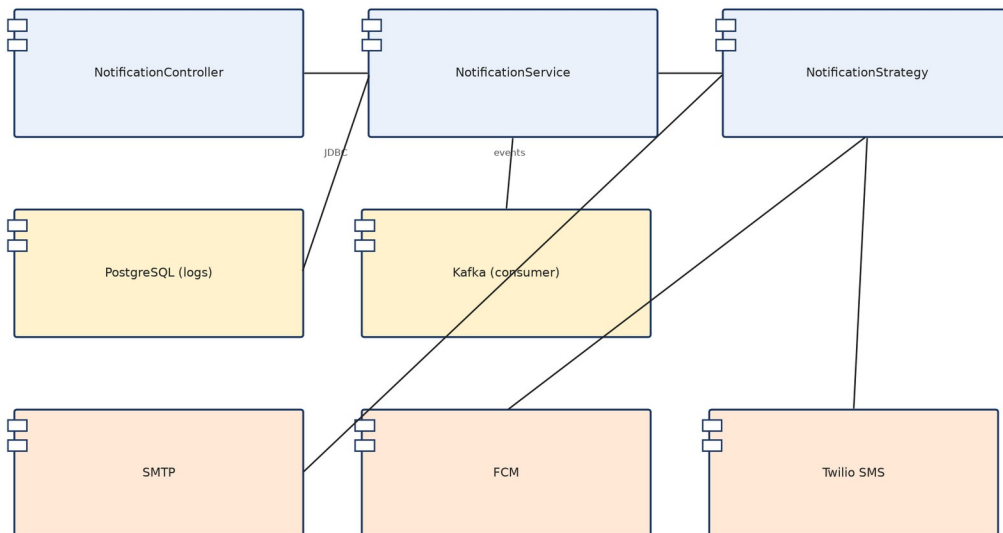
7.5.5 Report Card Service

Diagramme de Composants - Report Card Service



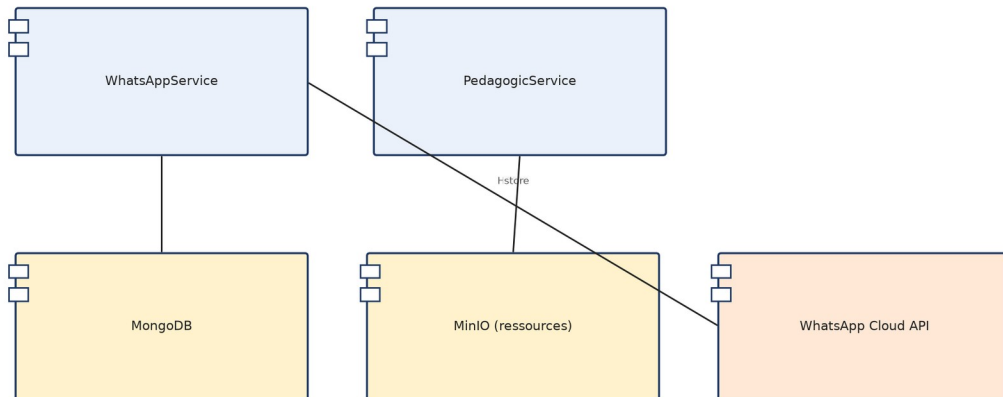
7.5.6 Notification Service

Diagramme de Composants - Notification Service



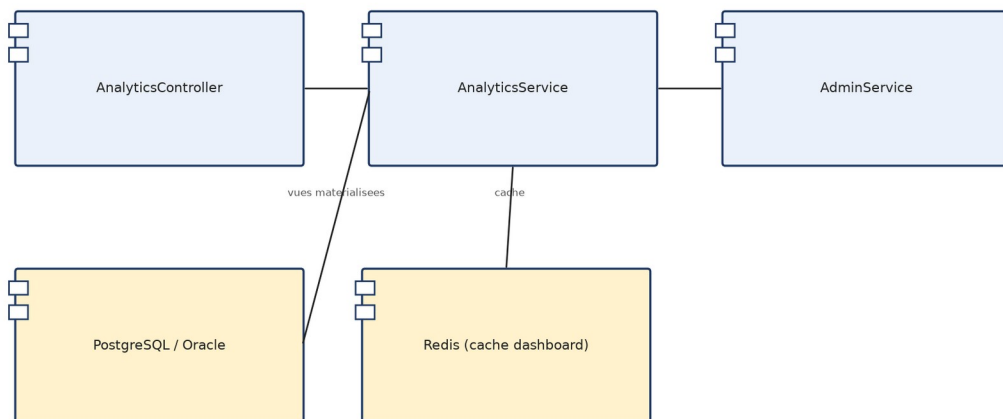
7.5.7 WhatsApp Community & Pedagogic Service

Diagramme de Composants - WhatsApp Community & Pedagogic Service



7.5.8 Analytics & Config/Admin Service

Diagramme de Composants - Analytics & Config/Admin Service



8. Reference Technique des Services

8.1 Conventions de Nommage

Toutes les ressources du projet SchoolManage suivent une convention de nommage uniforme basee sur le prefixe sm, le domaine fonctionnel et le type de ressource.

Type de ressource	Pattern de nommage	Exemples
Services Spring Boot	sm-{domaine}-service	sm-payment-service, sm-auth-service
Bases de donnees	sm-{domaine}-db	sm-payment-db, sm-auth-db
Topics Kafka	sm.{domaine}.{evenement}	sm.payment.completed, sm.presence.recorded
Cles Redis	sm:{domaine}:{objet}:{id}	sm:presence:lock:{studentId}
Buckets S3/MinIO	sm-{domaine}-{env}	sm-documents-prod, sm-reportcards-prod
Images Docker	sm/{service}:{version}	sm/payment-service:1.0.0
Variables d'environnement	SM_{DOMAINE}_{PARAM}	SM_PAYMENT_DB_URL

8.2 Micro-service et Ports

Chaque micro-service SchoolManage expose un port unique et fixe, utilise en developpement, en test et en production (via Docker / Kubernetes).

Nom du service	Port	Technologie	Role
sm-config-service	8080	Spring Cloud Config	Configuration centralisee
sm-registry-service	8761	Spring Cloud Eureka	Registre et decouverte de services
sm-gateway-service	8888	Spring Cloud Gateway	API Gateway - routage, JWT, RBAC
auth-service	8081	Spring Boot + Keycloak	Authentification, JWT, gestion des roles
registration-service	8082	Spring Boot	Inscription eleves / staff
payment-service	8083	Spring Boot	Paieement Mobile Money, transactions
presence-service	8084	Node.js / Express	Presence, check-in QR
notification-service	8085	Node.js / Express	Emails, SMS, notifications push
reportcard-service	8086	Python FastAPI	Notes, bulletins
userstatus-service	8087	Node.js / WebSocket	Statuts utilisateurs temps reel
whatsapp-service	8088	Node.js	Communaute WhatsApp
pedagogic-service	8089	Python FastAPI	Ressources pedagogiques
schoolid-service	8090	Python	Cartes d'identite scolaires
analytics-service	8091	Python	Tableaux de bord, KPI
admin-service	8092	Spring Boot	Configuration multi-etablissement

Ports infrastructure

Composant	Port	Usage
Redis	6379	Cache, verrous, sessions
PostgreSQL	5432	Base de donnees principale (par service)
PostgreSQL replica	5433	Replique lecture seule (reporting)
Kafka / RabbitMQ	9092 / 5672	Broker de messages
MinIO / S3	9000 (HTTPS)	Stockage fichiers : documents, recus, bulletins, cartes ID
Prometheus	9090	Collecte des metriques
Grafana	3000	Tableaux de bord de supervision

8.3 Bases de Donnees

Chaque micro-service possede sa propre base de donnees PostgreSQL (pattern Database per Service). Aucun service ne partage sa base avec un autre - la communication se fait uniquement via les API ou Kafka/RabbitMQ.

Nom de la base	Service propriétaire	Tables principales
sm-auth-db	auth-service	users_credentials, refresh_tokens, blacklisted_tokens, roles
sm-registration-db	registration-service	students, staff, registrations, documents
sm-payment-db	payment-service	payments, transactions, receipts, idempotency_keys
sm-presence-db	presence-service	attendance, checkin_events, presence_reports
sm-reportcard-db	reportcard-service	grades, subjects, report_cards
sm-notification-db	notification-service	notifications, templates, logs_envoi
sm-userstatus-db	userstatus-service	user_status, status_history
sm-schoolid-db	schoolid-service	school_ids, qr_codes
sm-admin-db	admin-service	establishments, school_configs

Bases complementaires : Oracle (rapports d'entreprise, archives financieres pluriannuelles, vues materialisees pour analytics-service) - MongoDB (archives des messages whatsapp-service).

8.4 Redis : Cles et Namespaces

Redis est utilise comme cache distribue, gestionnaire de verrous et stockage de metriques temps reel. Toutes les cles suivent la convention sm:{domaine}:{objet}:{id}.

Pattern de cle	Service propriétaire	TTL	Description
sm:auth:blacklist:{token}	auth-service	= exp JWT	Tokens JWT revoques (blacklist)
sm:auth:session:{userId}	auth-service	7 jours	Session active / refresh token
sm:presence:lock:{studentId}	presence-service	5 min	Verrou anti-doublon de check-in
sm:presence:today:{userId}	presence-service	24h	Statut de presence du jour
sm:payment:idempotency:{key}	payment-service	24h	Cle d'idempotence de paiement
sm:notification:rate:{userId}	notification-service	1h	Rate limiting des notifications
sm:analytics:dashboard:{schoolId}	analytics-service	1h	Cache du tableau de bord
sm:userstatus:{userId}	userstatus-service	Permanent	Statut courant de l'utilisateur

8.5 Kafka / RabbitMQ (Topics et Queues)

Kafka assure la communication asynchrone entre les micro-services. Chaque domaine possede son propre topic ; les queues sont liees par routing keys.

Topic / Queue	Consommateur(s)	Declencheur / Action
sm.auth.user.registered	notification-service, registration-service	Inscription -> email + creation profil
sm.registration.student.enrolled	schoolid-service, whatsapp-service, notification-service	Eleve inscrit -> ID + groupe WhatsApp + notif
sm.payment.completed	notification-service, analytics-service	Paieement reussi -> recu + KPI
sm.payment.failed	notification-service	Paieement echoue -> alerte parent
sm.presence.absence.detected	notification-service	Absence detectee -> alerte parent
sm.reportcard.available	notification-service	Bulletin genere -> notification
sm.userstatus.changed	notification-service	Changement de statut -> notification
sm.schoolid.generated	notification-service	Carte ID generee -> notification

sm.dead.letter	logging-service	Messages non traites apres 3 tentatives
----------------	-----------------	---

8.6 Environnement et Deploiement

Env	Nom	Infrastructure	Usage
DEV	Developpement local	Docker Compose	Chaque developpeur sur sa machine
TEST	Tests d'integration	K8s namespace test	Pipeline CI/CD - tests automatises
PROD	Production	K8s cluster multi-AZ	Deploiement final, haute disponibilite

9. Deploiement, Conteneurisation et Monitoring

9.1 Conteneurisation avec Docker

Chaque micro-service SchoolManage est packages dans une image Docker independante, construite a partir d'un Dockerfile multi-stage pour minimiser la taille finale : la premiere etape compile le code (Maven/pip/npm), la deuxieme ne conserve que l'artefact executable.

Service	Image de base	Port expose
sm-config-service / sm-registry-service / sm-gateway-service	eclipse-temurin:17-jre-alpine	8080 / 8761 / 8888
auth-service, registration-service, payment-service, admin-service	eclipse-temurin:17-jre-alpine	8081-8083, 8092
presence-service, notification-service, userstatus-service, whatsapp-service	node:20-alpine	8084-8085, 8087-8088
reportcard-service, pedagogic-service, schoolid-service, analytics-service	python:3.11-slim	8086, 8089-8091

En developpement, tous les services sont orchestres via un fichier docker-compose.yml unique. L'infrastructure partagee (PostgreSQL, Redis, Kafka) est declaree dans un compose.infra.yml separe pour pouvoir etre demarree independamment.

9.2 Orchestration avec Kubernetes

En production, la plateforme est deployee sur un cluster Kubernetes. Chaque service est represente par un Deployment associe a un Service (ClusterIP). Seul le sm-gateway-service est expose via un Ingress public.

Ressource Kubernetes	Role	Detail
Namespace schoolmanage-prod	Isolation	Tous les workloads dans un namespace dedie
Deployment (x15)	Execution des services	1 Deployment par micro-service, replicas configurables
Service ClusterIP (x15)	Communication interne	Resolution DNS interne : sm-payment-service:8083
Ingress (x1)	Point d'entree public	Expose uniquement le gateway sur le domaine SchoolManage
ConfigMap / Secret	Configuration / secrets	Variables non sensibles / JWT secret, cles Mobile Money
HorizontalPodAutoscaler	Auto-scaling	Scale automatique sur CPU > 70% - min 2 / max 10 pods
PersistentVolumeClaim	Persistence	Volumes pour PostgreSQL, Redis et Kafka
Liveness / Readiness Probe	Health checks	Via /actuator/health - redemarrage automatique si KO

Strategie de deploiement : Rolling Update par default (mise a jour progressive sans interruption de service), rollback automatique si les health checks echouent, HPA sur payment-service et presence-service (services a forte charge), namespaces separes dev/staging/prod.

9.3 Monitoring et Observabilite

La stack d'observabilite SchoolManage repose sur trois piliers : metriques (Prometheus + Grafana), traces distribuees (Zipkin/Jaeger), et logs centralises (stack ELK).

Metrique surveillee	Description
sm_requests_total	Nombre de requetes par endpoint et statut HTTP
sm_response_time_seconds	Temps de reponse P50 / P95 / P99 par service
sm_payments_completed_total	Compteur de paiements reussis (metrique metier)

sm_absences_detected_total	Nombre d'absences detectees par jour
redis_connected_clients	Nombre de connexions Redis actives
kafka_consumer_lag	Retard de consommation des topics Kafka (alerte si > 1000)

Alerte	Condition	Severite	Action
ServiceDown	health check KO > 30s	Critique	Restart automatique Kubernetes
HighResponseTime	P95 > 2s pendant 5 min	Eleve	Notification equipe + analyse traces
KafkaConsumerLagHigh	lag > 1000 pendant 2 min	Eleve	Scale consumer + investigation
PaymentWebhookFailed	5 echecs webhook consecutifs	Critique	Alerte equipe + verification idempotence
PodCrashLooping	pod restart > 3 fois en 5 min	Critique	Rollback automatique Kubernetes

9.4 Pipeline CI/CD

Le depot GitHub school-manage-monorepo est associe a un pipeline GitHub Actions declenche a chaque push sur main.

Etape	Action	Outil	Condition d'echec
1 - Build	Compilation + packaging de l'image Docker	Maven / pip / npm	Erreur de compilation
2 - Test	Tests unitaires et d'integration	JUnit / Jest / PyTest	Test KO
3 - Scan	Analyse des vulnerabilites de l'image	Trivy / OWASP Scan	CVE critique detectee
4 - Push	Publication de l'image sur le registry	GitHub Container Registry	Auth echouee
5 - Deploy	kubectl apply sur le cluster staging	kubectl / Helm	Health check KO
6 - Smoke test	Test de disponibilite des endpoints critiques	curl	Endpoint KO
7 - Prod	Deploiement production (validation manuelle)	kubectl	Approbation refusee

10. Backlog Initial des User Stories

Les user stories ci-dessous constituent le backlog produit initial de la plateforme SchoolManage App, regroupees par epic.

Epic 1 : Inscription & Identite

- En tant qu'administrateur, je veux inscrire un eleve afin de creer son dossier scolaire.
- En tant qu'administrateur, je veux uploader les documents d'un eleve afin de completer son dossier.
- En tant que systeme, je veux generer automatiquement une carte ID afin d'identifier l'eleve.
- En tant qu'administrateur, je veux affecter une classe a un eleve afin d'organiser les effectifs.

Epic 2 : Paiement

- En tant que parent, je veux payer les frais de scolarite en ligne afin d'eviter le deplacement en agence.
- En tant que parent, je veux recevoir un reçu automatique afin de justifier mon paiement.
- En tant qu'administrateur, je veux payer les salaires du personnel afin d'automatiser la paie.
- En tant que parent, je veux consulter l'historique de mes paiements afin de suivre mes depenses.

Epic 3 : Presence

- En tant qu'eleve, je veux scanner mon QR code afin d'enregistrer ma presence.
- En tant qu'enseignant, je veux consulter le tableau de presence afin de suivre l'assiduite de ma classe.
- En tant que parent, je veux etre alerte en cas d'absence non justifiee afin de reagir rapidement.

Epic 4 : Pedagogie & Bulletins

- En tant qu'enseignant, je veux saisir les notes afin d'evaluer mes eleves.
- En tant qu'administrateur, je veux generer les bulletins afin de cloturer une periode scolaire.
- En tant que parent, je veux consulter le bulletin de mon enfant afin de suivre sa scolarite.

Epic 5 : Communication & Supervision

- En tant qu'enseignant, je veux diffuser une annonce afin d'informer les parents d'une classe.
- En tant que directeur, je veux consulter le tableau de bord afin de piloter mon etablissement.
- En tant qu'administrateur systeme, je veux ajouter un etablissement afin d'etendre la plateforme.
- En tant qu'administrateur systeme, je veux desactiver un compte afin de gerer les abus.

11. Exigences Non Fonctionnelles

11.1 Performance

Exigence	Cible	Mecanisme
Temps de reponse API (P95)	< 200 ms	Cache Redis, PostgreSQL read replicas
Temps de reponse paiement (init)	< 2 s	Idempotence Redis, traitement asynchrone
Disponibilite plateforme (SLA)	> 99,9 %	Architecture micro-services, load balancing
Utilisateurs simultanés supportés	> 10 000	Scalabilite horizontale par service (K8s HPA)
Generation d'un bulletin PDF	< 5 s	Traitement asynchrone, cache 1h
Chargement application mobile	< 3 s	CDN, pagination, lazy loading

11.2 Securite

- Authentification JWT avec expiration courte (15 min) et refresh token (7 jours)
- Blacklist Redis des tokens invalides pour revocation immediate
- RBAC strict : 4 roles avec permissions encodees dans le payload JWT
- HTTPS obligatoire sur tous les endpoints publics et webhooks
- Validation et sanitisation de toutes les entrees utilisateur (OWASP Top 10)
- Rate limiting sur l'API Gateway (anti brute-force)
- Cles d'idempotence pour les paiements Mobile Money
- Audit complet des transactions financieres en base PostgreSQL/Oracle
- Conformite RGPD (traitement des donnees et droit a l'effacement)

11.3 Maintenabilite

- Architecture micro-services : deploiement independant de chaque service
- Communication asynchrone via Kafka : decouplage fort entre services
- Logs structures par service avec niveaux INFO / WARNING / ERROR
- Tests unitaires et d'integration par service
- Documentation API OpenAPI/Swagger par service

11.4 Evolutivite

- Ajout de nouveaux modes de paiement (Wave, cartes bancaires) sans modifier les autres services
- Ajout de nouveaux etablissements sans deploiement global
- Scalabilite horizontale : chaque micro-service peut etre replique independamment
- Ajout de nouveaux canaux de notification (WhatsApp Business, Viber) via le pattern Strategy

12. Glossaire

Terme	Définition
API Gateway	Point d'entrée unique qui route les requêtes clients vers les micro-services appropriés.
RBAC	Role-Based Access Control - contrôle d'accès basé sur les rôles utilisateurs.
JWT	JSON Web Token - jeton signé utilisé pour l'authentification et l'autorisation.
Idempotence	Propriété garantissant qu'une même opération exécutée plusieurs fois produit le même résultat (évite les doubles débits).
Pattern Saga	Séquence de transactions locales coordonnées par des événements, utilisée pour maintenir la cohérence entre micro-services.
Database per Service	Principe architectural où chaque micro-service possède sa propre base de données isolée.
Service Discovery	Mécanisme permettant aux services de se localiser dynamiquement (Eureka).
Circuit Breaker	Mécanisme qui isole automatiquement un service défaillant pour éviter la propagation des pannes.
Webhook	Requête HTTP envoyée automatiquement par un système externe (ex: opérateur Mobile Money) pour notifier un événement.
KPI	Key Performance Indicator - indicateur clé de performance.
TTL	Time To Live - durée de vie d'une donnée en cache avant expiration.
Rolling Update	Stratégie de déploiement mettant à jour les instances progressivement sans interruption de service.

13. Conclusion

SchoolManage App propose une reponse structuree et evolutive aux besoins de digitalisation des etablissements scolaires camerounais. Son architecture micro-services - 12 services metier et 3 services d'infrastructure - garantit une independance de deploiement et une scalabilite adaptees a une croissance progressive du nombre d'etablissements et d'utilisateurs.

La conception presentee dans ce document - acteurs et cas d'utilisation detailles, description complete des 15 micro-services (endpoints, communications, donnees), diagrammes UML (classes, sequence, activite, etat-transition, composants), reference technique (nommage, ports, Redis, Kafka), strategie de deploiement Kubernetes et plan de tests - constitue la base technique sur laquelle s'appuiera le developpement de la plateforme.

Les prochaines etapes consistent a valider ce dossier de conception aupres des parties prenantes, puis a entamer le developpement selon le planning etabli, en commençant par les services fondateurs (auth-service, registration-service) avant d'etendre progressivement aux modules avances (analytics, whatsapp, pedagogic).